

Kharcha

AI-native Expense & Reimbursement Agent

From a pile of receipts to "ready to submit" in under two minutes, with on-device AI that reads, checks and verifies every bill before it leaves the phone.



PREPARED BY	VERSION	DATE	SCOPE
Rahul	1.0 Draft for build	7 October 2026	Android app, backend, mock finance

CONTENTS

What is in this document

How to read this

Part A, the PRD. Sections 1 to 4 cover the problem, users and success metrics. Sections 5 to 9 describe the experience, requirements, policy and per-employee entitlements. Sections 10 to 15 cover the AI pipeline, authenticity checks, architecture, model providers, data and APIs. Sections 16 to 19 cover quality, the build plan, risks and open questions. **Part B, the development guide.** Sections 20 to 28 give class design, the database schema, Oracle Cloud and Docker setup, provider keys, CI/CD and a day-by-day plan; Sections 29 and 30 and Appendix A make the document executable by Claude Code across the laptop and the Oracle VM. Requirement priorities: **P0** must ship in the prototype, **P1** should ship, **P2** is stretch.

1. Executive summary

2. Problem and opportunity

3. Goals, non-goals and success metrics

4. Users and personas

5. The experience: pile to ready

6. Functional requirements

7. Asking only what is necessary

8. Sample policy for the prototype

9. Employee entitlements and claim limits

10. AI pipeline and on-device models

11. Authenticity and fraud signals

12. System architecture

13. Model provider and LLM gateway

14. Data model

15. APIs and claim lifecycle

16. Non-functional requirements

17. Build plan and demo

18. Risks and mitigations

19. Open questions and future scope

Part B | Development guide

20. Engineering setup and repository

21. Android app design

22. Backend design

23. Database design

24. Oracle Cloud setup

25. Installing the stack

26. NVIDIA model setup

27. CI/CD, testing and operations

4

5

5

6

7

9

10

11

11

14

16

17

18

20

20

21

22

23

23

25

26

28

31

33

36

38

41

44

28. Day-by-day development plan

29. Building with Claude Code

30. Two-workspace development

A. Appendix: task board

45

46

49

52

THE SHORT VERSION

1

Executive summary

Filing an expense claim is low-value work that every employee does and nobody enjoys. People collect paper and digital bills, retype amounts and dates, guess the right category, check a policy they half remember, and then wait for a claim to bounce back because a GST number was missing or a hotel exceeded a cap. Finance teams then repeat much of the same checking by hand.

Kharcha is an AI-native reimbursement assistant on Android with a chat-style interface. The employee drops in photos, PDFs, screenshots or forwarded e-bills. On-device AI reads each document, extracts structured fields, checks whether the document looks genuine, applies policy rules, spots duplicates, groups expenses into trips or categories, asks only the questions it truly cannot answer itself, and produces a claim that is ready to submit to a (mocked) finance system.

<2 min

User time from 10 receipts to a submit-ready claim (target)

≤2

Median questions asked per claim

≥95%

Field accuracy on amount, date, vendor (golden set)

-60%

Finance review time per claim (target)

Why this design

- **On-device first.** Receipts carry personal data (names, phone numbers, card digits, locations). Reading them on the phone keeps raw images local until the user submits, works offline in a cab or airport, and removes per-receipt cloud inference cost.
- **Agent, not form.** The assistant owns the claim. It tracks what is complete, missing or questionable and drives the conversation to "ready" with the fewest possible taps.
- **Trust is explainable.** Every flag (possible edit, duplicate, policy breach) carries evidence and a plain-language reason. The system never auto-rejects; people decide.
- **Mock the edges, build the core.** The finance/ERP system, SSO and payouts are mocked so the build effort goes into extraction, verification and the experience.

One-line pitch

Kharcha turns a pile of receipts into a verified, policy-checked, submit-ready claim in under two minutes, and gives finance a claim they can approve without re-checking every bill.

2

WHY THIS MATTERS

Problem and opportunity

Where time and money leak today

Stage	What happens today	Cost of the problem
Collecting	Paper bills fade or get lost; e-bills are scattered across email, cab apps and WhatsApp.	Claims filed late or never; out-of-pocket loss for employees.
Data entry	Employee retypes vendor, date, amount, GST, category for each bill.	Typically 2 to 4 minutes per receipt; typos in amounts and dates.
Policy	Employee guesses limits; policy PDF is long and rarely read.	Claims returned for avoidable reasons; frustration on both sides.
Verification	Finance eyeballs each bill for edits, duplicates and missing GSTIN.	Slow approvals; inconsistent checks; fraud and duplicates slip through.
Follow-up	Back-and-forth emails for missing details, often days later.	Long cycle time; context lost; poor employee experience.

Illustrative value model

The numbers below are planning assumptions for a mid-sized organisation, not measured data. The prototype will measure actual time saved on a synthetic test set and replace these assumptions.

Assumption	Value	Notes
Employees filing claims	1,000	Sales, field and travelling staff file most claims
Claims per employee per month	2	Average of ~5 receipts per claim
Employee time saved per claim	20 min	From ~23 min manual to ~3 min with Kharcha
Finance time saved per claim	6 min	Pre-verified fields, evidence-backed flags
Loaded cost per hour	₹600	Blended assumption
Monthly hours saved	~870 h	667 h employees + 200 h finance
Monthly value	~₹5.2 lakh	Before leakage recovered from duplicates and over-limit claims

3

WHAT WE WILL AND WILL NOT DO

Goals, non-goals and success metrics

Goals

- G1. Minimise user effort from receipts to a submit-ready claim, measured in time and taps.
- G2. Produce structured, reliable claim data with field-level confidence and evidence.
- G3. Detect likely tampered, recaptured or duplicate documents and explain why.
- G4. Apply policy before submission so claims are right first time.
- G5. Keep personal data on the device by default and degrade gracefully on low-end phones.

Non-goals for the prototype

- Integration with a real ERP or payroll system (a mock finance API stands in).
- Corporate card feed reconciliation, multi-currency FX and per-country tax regimes beyond India GST.
- Final fraud determination. Kharcha raises risk signals; humans decide.
- iOS app and full web app for employees (a light approver console is in scope).

Success metrics

Metric	Definition	Target	How measured
Time to ready	User active time from first drop to "Ready to submit" for 10 receipts	< 2 min	Instrumented timestamps in app
Questions per claim	Agent questions that need a user answer	Median ≤ 2	Agent event log
Key-field accuracy	Exact match on total, date, vendor, GSTIN	≥ 95%	Golden set of 300 documents
Category accuracy	Correct expense category vs label	≥ 92%	Golden set
Tamper detection	Recall on tampered set at ≤ 10% false positives	≥ 80% recall	Synthetic tampered variants
Duplicate detection	Recall on planted duplicates	≥ 95%	Planted pairs in dataset
First-pass approval	Claims approved without being returned	+30 pts vs baseline	Mock approver flow
Over-limit leakage	Over-limit amounts submitted without a flag or exception	0	Planted over-limit scenarios
On-device latency	Per-receipt extraction, mid-range phone	p50 < 3 s	Benchmark harness

4

WHO WE ARE BUILDING FOR

Users and personas

Persona	Context	Needs from Kharcha
Priya, field sales executive	Travels 3 to 4 days a week. 30+ small receipts a month: cabs, meals, tolls, fuel, hotel. Files claims in batches on Sunday nights.	Wants one place to throw everything and be told what is missing. Hates retyping and getting claims returned.
Arjun, software engineer	Claims rarely: broadband, an offsite hotel, a conference ticket. Does not remember the policy.	Wants it to just work and to know upfront whether something is reimbursable.
Meera, approving manager	Approves 40 claims a month for her team, mostly on mobile between meetings.	Wants a short summary with anything unusual highlighted and the evidence one tap away.
Vikram, finance reviewer	Audits claims before payout. Looks for duplicates, edits, missing GST invoices, policy breaches.	Wants structured data, consistent checks and an audit trail he can defend.

Jobs to be done

- "When I get back from a trip, help me turn everything I collected into one correct claim without retyping anything."

- "Tell me before I submit if something will be rejected, and what I can do about it."
- "When I approve, show me only what needs my judgement, with proof."

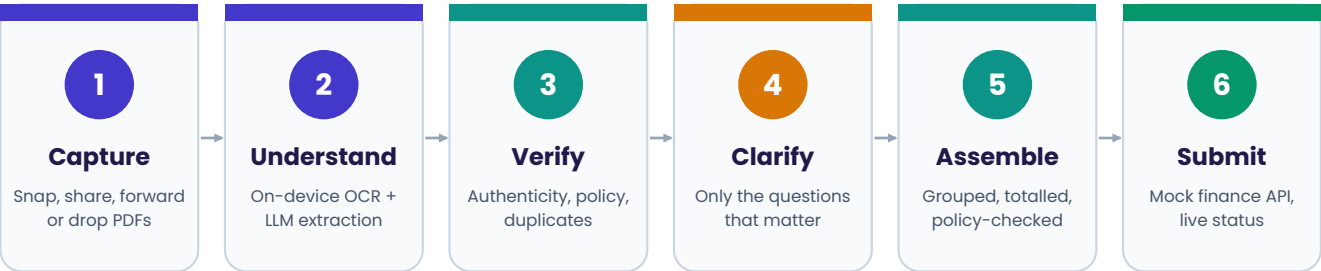
5

USER-FACING CHALLENGE

The experience: pile to ready

The core interaction is a chat thread with a drop zone. Everything else (cards, checklists, chips) appears inside that thread so the user never has to learn a form. Processing starts the instant a file lands; the user can keep dropping more while earlier receipts are being read.

END-TO-END JOURNEY



Target: pile of 10 receipts to ready-to-submit claim in under 2 minutes of user time

Figure 1. Six stages; the user is only actively involved in Capture, Clarify and Submit.

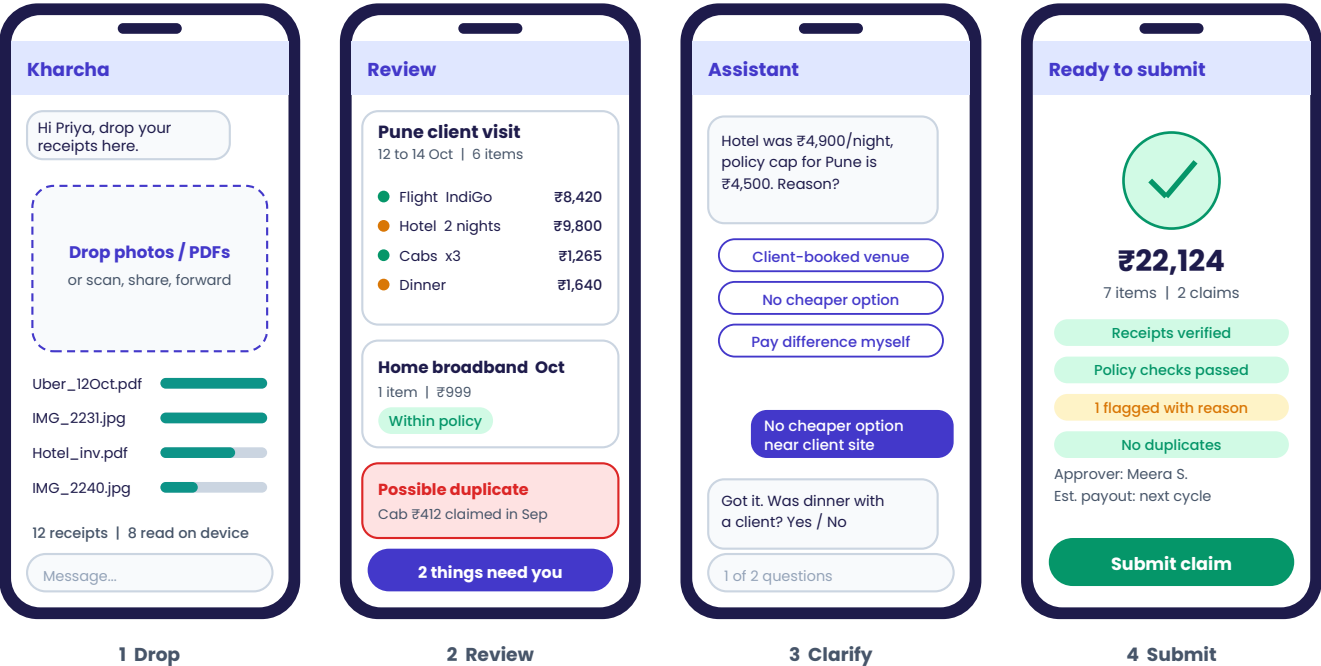


Figure 2. Low-fidelity wireframes of the four key screens.

Experience principles

Principle	What it means in the product
Do the work, then ask	Extract, infer and default everything possible before asking. A question is a last resort, not a form field.
Ask once per context	Business purpose is asked once per trip, not per receipt. Attendees are asked only where policy requires them.
One-tap answers	Every question offers suggested answers as chips, with free text as a fallback. Voice input is supported.
Show status, not steps	Each receipt and claim shows a clear state: Done, Needs you, Flagged. A single counter says how many things need attention.
Explain every flag	"We could not verify this hotel invoice: the PDF was modified after it was issued" rather than "Suspicious document".
Non-accusatory tone	Flags are framed as checks the system could not complete, giving the employee a chance to explain or re-upload.
Always reversible	Every auto-filled value is editable in place; the original document and highlighted source region are one tap away.

Capture channels

Channel	Behaviour	Pri
Camera scan	ML Kit Document Scanner with auto edge detection, deskew and multi-page capture.	P0
Gallery / file picker	Multi-select images and PDFs; bulk drop of 20+ files.	P0
Android share sheet	Share a cab receipt PDF or screenshot from any app straight into Kharcha.	P0
Email forward	Forward e-bills to a personal claims address; backend parses attachments (mocked mailbox).	P2
Long receipts	Stitch multiple photos of a long thermal receipt into one document.	P2

6

WHAT THE SYSTEM MUST DO

Functional requirements

6.1 Ingestion and understanding

ID	Requirement	Pri	Acceptance criteria
ING-1	Accept JPG, PNG, HEIC, WEBP and PDF (single and multi-page), up to 20 files per drop.	P0	20 mixed files queue without blocking UI; progress shown per file.
ING-2	Run a quality gate for blur, glare, cut-off edges and low resolution before OCR.	P0	Blurred test images trigger a retake prompt within 1 s.
ING-3	Use the embedded text layer for digital PDFs; OCR only when it is missing.	P0	Digital PDFs skip OCR; text matches source.
EXT-1	Classify each document: cab, flight, rail, hotel, meal, fuel, toll, telecom, other.	P0	≥ 92% category accuracy on golden set.
EXT-2	Extract vendor, date, time, total, tax lines, GSTIN, invoice number, payment mode, city, line items.	P0	Fields returned as typed JSON with per-field confidence and source box.
EXT-3	Handle multiple receipts in one photo and multi-receipt PDFs by splitting them.	P1	Two receipts in one image become two expense lines.
EXT-4	Support English and Hindi text; mixed scripts on the same bill.	P1	Hindi vendor names extracted on test set.
EXT-5	Normalise values: dates to ISO, amounts to paise integers, vendor names to canonical form.	P0	"Ola Cabs", "OLA" and "ANI Technologies" map to one vendor.

6.2 Verification and policy

ID	Requirement	Pri	Acceptance criteria
VER-1	Compute a trust score (0 to 100) per document from authenticity signals (Section 11).	P0	Score and top reasons visible on each receipt card.
VER-2	Validate GSTIN format and checksum; validate tax arithmetic (CGST + SGST or IGST vs total).	P0	Bad checksum or mismatched math raises a flag with the numbers shown.
VER-3	Detect duplicates within a claim, across the user's past claims and across employees.	P0	Planted duplicates flagged with link to the original claim.
POL-1	Evaluate each line against policy rules held as data (limits, eligibility, receipts needed).	P0	Rule changes in JSON apply without an app release.
POL-2	Show the reason and the rule for every breach, and offer remedies (justify, split, pay difference).	P0	Each breach has rule ID, limit, actual and remedy options.
POL-3	Show "will this be reimbursed?" before submission for any single receipt.	P1	Answer appears within 1 s of extraction.
ENT-1	Check every line against the employee's own per-category and overall limits fetched from the (mock) ERP.	P0	Exhausted, partly covered and not-entitled states shown per line (Section 9).

6.3 Agent, grouping and claim assembly

ID	Requirement	Pri	Acceptance criteria
AGT-1	Group expenses into trips or categories using dates, cities and travel bookings.	P0	Flight + hotel + cabs in same city and dates become one trip.
AGT-2	Track claim completeness: complete, missing, questionable, for every line and the claim.	P0	A single "N things need you" counter that reaches zero.
AGT-3	Ask only questions required by policy or by low confidence; batch them; offer chips.	P0	Median ≤ 2 questions per claim on test scenarios.
AGT-4	Accept natural-language edits ("the dinner was with two clients from Acme").	P1	Attendee and purpose fields updated correctly.
CLM-1	Generate a submission-ready claim with summary, lines, attachments and flag notes.	P0	Claim JSON passes mock finance schema validation.
CLM-2	Submit to the mock finance API and show live status (submitted, in review, approved, paid).	P0	Status changes from mock API appear in the thread.
CLM-3	Approver console: summary, flags with evidence, approve / return / reject with comment.	P1	Returned claim reopens in the employee's thread with the comment.
CLM-4	Export claim as a PDF report with receipts appended.	P2	PDF includes summary table and all images.

THE QUESTION POLICY

7

Asking only what is necessary

The agent decides whether to ask a question with a simple, inspectable rule rather than leaving it to the LLM. A question is asked only when a field is **required** (by policy or by the finance schema) **and** it is either missing or below its confidence threshold **and** it cannot be inferred from other documents in the same claim.

```
ask(field) = required(field, line, policy)
              AND (missing(field) OR confidence(field) < threshold(field))
              AND NOT inferable(field, claim_context)

priority = blocking_submit * 3 + policy_breach * 2 + low_confidence * 1
batch    = group questions by trip / context; max 3 per turn
```

Situation	Asked?	Example question
Total extracted at 0.98 confidence	No	(auto-filled)
Business purpose missing for a 3-day trip	Once per trip	"What was the Pune trip for?" [Client visit] [Training] [Internal meeting]
Meal above per-person limit	Yes	"Was this dinner with clients? How many people?" [Just me] [2] [3+]
Hotel above city cap	Yes	"Hotel was ₹4,900 a night vs ₹4,500 cap. Reason?" [Client venue] [No cheaper option] [I'll pay the difference]
Cab receipt city unclear but flight shows Pune	No	(inferred from trip context)
Total at 0.62 confidence, smudged	Yes, with crop	"Is the total ₹1,640?" shown with highlighted crop [Yes] [Edit]

RULES AS DATA

8

Sample policy for the prototype

The prototype ships with a synthetic policy so rules can be demonstrated end to end. Rules are stored as JSON and evaluated by the backend policy engine; a cached copy runs on device for instant feedback.

Rule ID	Category	Rule	On breach
P-HTL-1	Hotel	Tier-1 metro ≤ ₹6,000 / night; other cities ≤ ₹4,500 / night	Justify or pay difference
P-MEAL-1	Meals	≤ ₹1,000 per person per day when travelling; client meals need attendees	Justify; attendees required
P-CAB-1	Local travel	Economy cab classes only; premium class needs a reason	Justify
P-GST-1	All	Bills above ₹5,000 need a GST invoice with company GSTIN	Flag; request proper invoice
P-AGE-1	All	Receipts older than 60 days are not reimbursable	Block submit; route to manager exception
P-EXCL-1	All	Alcohol, fines, personal items and loyalty-point redemptions excluded	Exclude line; show reason
P-TEL-1	Telecom	Broadband needs a bill in the employee's name or address; amount capped by the employee's entitlement (Section 9)	Request proper bill; apply entitlement

```
{
  "id": "P-HTL-1",
  "applies_to": { "category": "hotel" },
  "limit": { "per": "night", "amount_paise": {
    "city_tier == 1": 600000, "default": 450000 } },
  "on_breach": { "severity": "amber",
    "remedies": ["justify", "pay_difference"], "requires": ["reason"] }
}
```

PER-PERSON BALANCES FROM FINANCE

9

Employee entitlements and claim limits

Policy rules (Section 8) apply to every employee and judge a single expense: is this hotel night too expensive? **Entitlements** are different. They are each employee's own budget for a category over a period, such as ₹15,000 a year for broadband or ₹4,000 a month for travel meals. They vary by grade, role, location and individual approvals, so Kharcha never hard-codes them. It reads them from the finance / ERP system (mocked for the prototype) and tracks how much is used.

EMPLOYEE ENTITLEMENTS (FROM MOCK ERP, FY 2026-27)

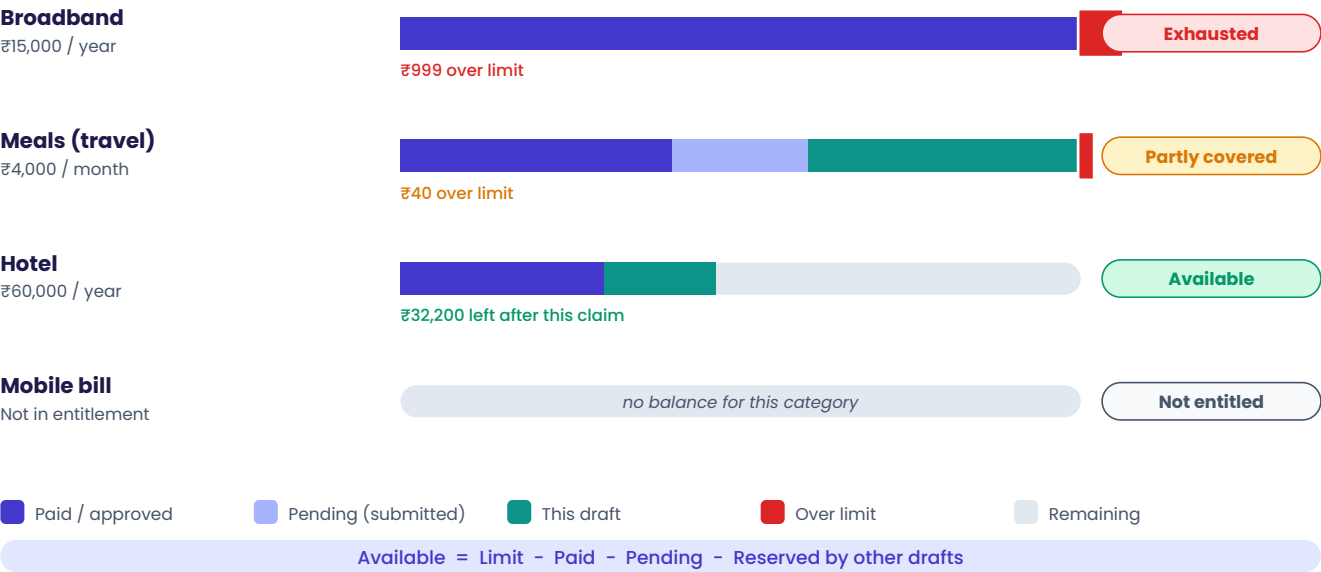


Figure 3. How a draft claim is checked against each employee's balances. Values are synthetic.

How the balance is calculated

For each category and period, the ERP supplies the limit and the amount already paid. Kharcha adds what it knows that the ERP may not yet show: claims submitted but not yet paid (pending) and amounts reserved by the employee's other open drafts. **Available = limit - paid - pending - reserved**. The server recalculates this at submit time with the latest ERP data; the value on the phone is a preview with an "as of" time.

Order of checks for each expense line

1. **Eligibility.** Is this category in the employee's entitlement at all? If not, the line is marked "Not entitled".
2. **Policy rule.** Company-wide rule for the single expense (cap per night, GST invoice, receipt age).
3. **Category entitlement.** Does the amount fit in the remaining category balance for the period of the expense date?
4. **Overall cap.** If the ERP sets a total reimbursement cap across categories, check that too.
5. **Reimbursable amount.** The lowest of the bill amount, the policy cap and the available balances becomes the claimable amount; the rest is shown as not reimbursable.

States shown to the employee

State	When	What the assistant says (example)	Options
Available	Amount fits with room left	"Broadband ₹999 is covered. ₹2,001 of ₹15,000 left this year."	None needed
Nearly used	80% or more of the limit used after this claim	"After this, only ₹400 is left for travel meals this month."	None needed
Partly covered	Bill is more than the remaining balance	"₹1,600 of this ₹1,640 dinner can be claimed; ₹40 is over your monthly meal limit."	Claim covered part / Request exception
Exhausted	Balance is zero for the period	"Your broadband limit of ₹15,000 for FY 2026-27 is fully used. It resets on 1 Apr 2027."	Remove / Keep for records / Request exception
Not entitled	Category not in the employee's entitlement	"Mobile bills are not part of your reimbursement plan."	Remove / Ask manager
Unknown	ERP unreachable and no cached data	"I could not check your balance; it will be checked when you submit."	Continue

Tell the user early, not at submit

The balance is shown as soon as a receipt is read, and the chat home shows a small "limits" strip for the categories the employee claims most. An exhausted category is mentioned before the employee spends time answering questions about that receipt. The assistant never asks for a business purpose on a line that cannot be reimbursed.

Requirements

ID	Requirement	Pri	Acceptance criteria
ENT-1	Fetch each employee's entitlements from the mock ERP: category, limit, period (month, quarter, financial year, per trip), period dates, paid amount, overall cap.	P0	Mock ERP returns different limits for different employees; app shows them.
ENT-2	Compute available balance including pending claims and reservations from other drafts.	P0	Two drafts cannot both use the same ₹2,000.
ENT-3	Show per-line state (available, nearly used, partly covered, exhausted, not entitled) with the reset date.	P0	Exhausted broadband shows the message and reset date.
ENT-4	Split a line into reimbursable and non-reimbursable parts when partly covered.	P0	Claim total equals the covered amount; excess listed separately.
ENT-5	Use the period that contains the expense date, not the submission date.	P0	A March bill submitted in April uses the March balance.
ENT-6	Re-check balances on the server at submit; if they changed, update the claim and tell the user before sending.	P0	Stale cache never lets an over-limit claim through unflagged.
ENT-7	Release reservations when a claim is returned, rejected or a line is deleted.	P1	Balance increases immediately in the app.
ENT-8	Exception request: send the over-limit part to the manager with a reason.	P1	Approver console shows exception separately from the main claim.
ENT-9	Cache entitlements on device with a fetched-at time for offline use; refresh at app open and before submit.	P1	Offline mode shows "as of" time.

Edge cases

- **Pro-rata limits** for joiners, leavers and grade changes come from the ERP; Kharcha does not calculate them.
- **Multi-period bills** (a quarterly broadband bill) are counted in the period of the bill date unless the ERP says otherwise.
- **Refunds and credit notes** reduce the paid amount and free up balance when the ERP reports them.
- **Same bill, two people** (shared hotel room) is a duplicate check, not an entitlement split, in the prototype.

BACKEND CHALLENGE, PART 1

10

AI pipeline and on-device models

Extraction is a hybrid of a generative model and deterministic code. The model reads messy layouts and normalises them into a strict schema; code then validates everything it can (checksums, arithmetic, dates) so the model is never the only source of truth for a number.

ON-DEVICE EXTRACTION PIPELINE

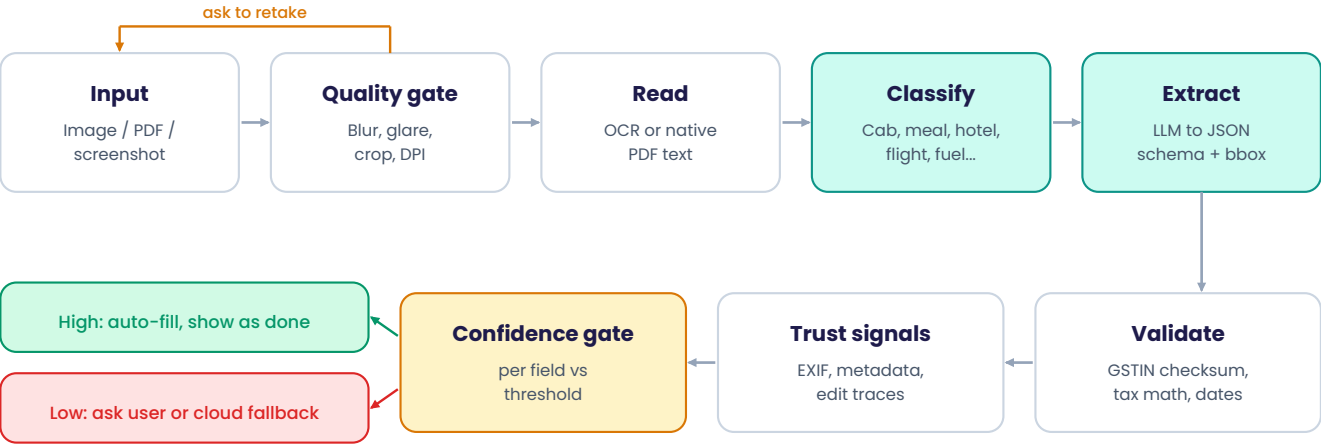


Figure 4. Each receipt flows through the same pipeline; low-confidence fields become questions or go to a cloud fallback.

Device tiering

On-device LLM support varies widely across Android hardware, so the app picks a path at runtime based on capability checks, available RAM and thermal state.

Tier	Device profile	Extraction path	Notes
A	Devices supporting ML Kit GenAI (Gemini Nano via AICore)	ML Kit Text Recognition v2 + Gemini Nano Prompt API with JSON-schema prompt	Best latency, no model download. Check feature availability at runtime; supported device list is limited.
B	≥ 6 GB RAM, recent SoC, no AICore	OCR + small open model (e.g. Gemma family, 1B-class int4) via LiteRT-LM	One-time model download (~0.5 to 1 GB). Run only when charging or thermal headroom allows for large batches.
C	Low-end or thermally throttled	On-device OCR only; redacted text sent to backend LLM	User consent required; PII redaction before upload.

Thermal and memory guardrails

Batch extraction checks the Android thermal status before each document and pauses or downgrades to Tier C when the device reports moderate or higher throttling. Only one model instance is kept in memory; OCR runs while the LLM is idle to keep peak memory low. These limits are measured in the benchmark harness on at least one mid-range device.

Extraction contract

The model is prompted with the OCR text (and, on multimodal tiers, the image) plus a strict JSON schema. Output that fails schema validation is retried once with the error message, then falls back.

```
{
  "doc_id": "d_8f21", "doc_type": "meal", "lang": ["en"],
  "vendor": { "value": "Barbeque Nation, Koregaon Park", "conf": 0.97, "bbox": [42,18,310,44] },
  "date": { "value": "2026-10-13", "conf": 0.99 },
  "total": { "value_paise": 164000, "currency": "INR", "conf": 0.96 },
  "tax": { "cgst_paise": 3905, "sgst_paise": 3905, "rate": 5, "conf": 0.91 },
  "gstn": { "value": "27ABCDE1234F1Z0", "checksum_ok": true },
  "invoice_no": { "value": "KP/26/04412", "conf": 0.88 },
  "payment_mode": { "value": "UPI", "conf": 0.80 },
  "line_items": [ { "desc": "Veg buffet x2", "amount_paise": 156190 } ],
  "trust": { "score": 86, "signals": ["camera_capture", "exif_consistent", "tax_math_ok"] }
}
```

Example values are synthetic.

Evaluation plan

- **Golden set:** 300 documents (200 synthetic generated from templates with randomised vendors, layouts, lighting and fold artefacts, plus 100 from public receipt datasets such as SROIE and CORD, subject to their licences).
- **Tampered set:** edited totals, altered dates, pasted logos, screenshots of screens, re-saved PDFs, and exact and near duplicates.
- **Metrics:** per-field exact match, category accuracy, tamper precision and recall, questions per claim, latency p50 and p95 per tier.
- **Regression:** evaluation runs in CI on every prompt or model change, with a results table committed to the repository.

11

IS THIS DOCUMENT GENUINE?

Authenticity and fraud signals

No single check proves a receipt is genuine. Kharcha combines many weak signals into a trust score and shows the strongest reasons. Cheap checks run on device at capture time; heavier forensics run on the backend after submission.

Signal	What it checks	Where	Weight
Capture source	Live camera scan vs gallery vs screenshot vs downloaded PDF	Device	Low
EXIF consistency	Capture time vs receipt date; editing software tags; missing EXIF on a "photo"	Device	Med
PDF metadata	Producer/creator tool, modification after creation, incremental updates, mixed fonts in text layer	Device	High
Arithmetic	Line items, tax and total add up; GST rate matches category	Device	High
GSTIN validity	Format, state code, checksum; (mock) lookup that GSTIN belongs to the vendor name	Both	High
Screen recapture	Moiré patterns and screen bezels suggesting a photo of a screen	Backend	Med
Error level analysis	JPEG recompression differences around the total or date region	Backend	Med
Template match	Layout compared with known vendor templates (cab apps, airlines, hotel chains)	Backend	Med
Duplicate	SHA-256 exact, perceptual hash near-duplicate, and vendor + date + amount fuzzy key	Backend	High

Signal	What it checks	Where	Weight
Behavioural	Round-number totals, sequences just under limits, many receipts from one small vendor	Backend	Low

TRUST SCORE BANDS (PER DOCUMENT)



Figure 5. Bands are tuned on the tampered set to keep false positives at or below 10%.

Fairness guardrail

Low-trust documents are never auto-rejected. Thermal paper fades, small vendors issue handwritten bills, and genuine PDFs are sometimes re-saved. The employee always sees why a document was flagged and can add a note, upload a better copy or attach supporting evidence. Every override is recorded in the audit log.

12

HOW THE PIECES FIT

System architecture

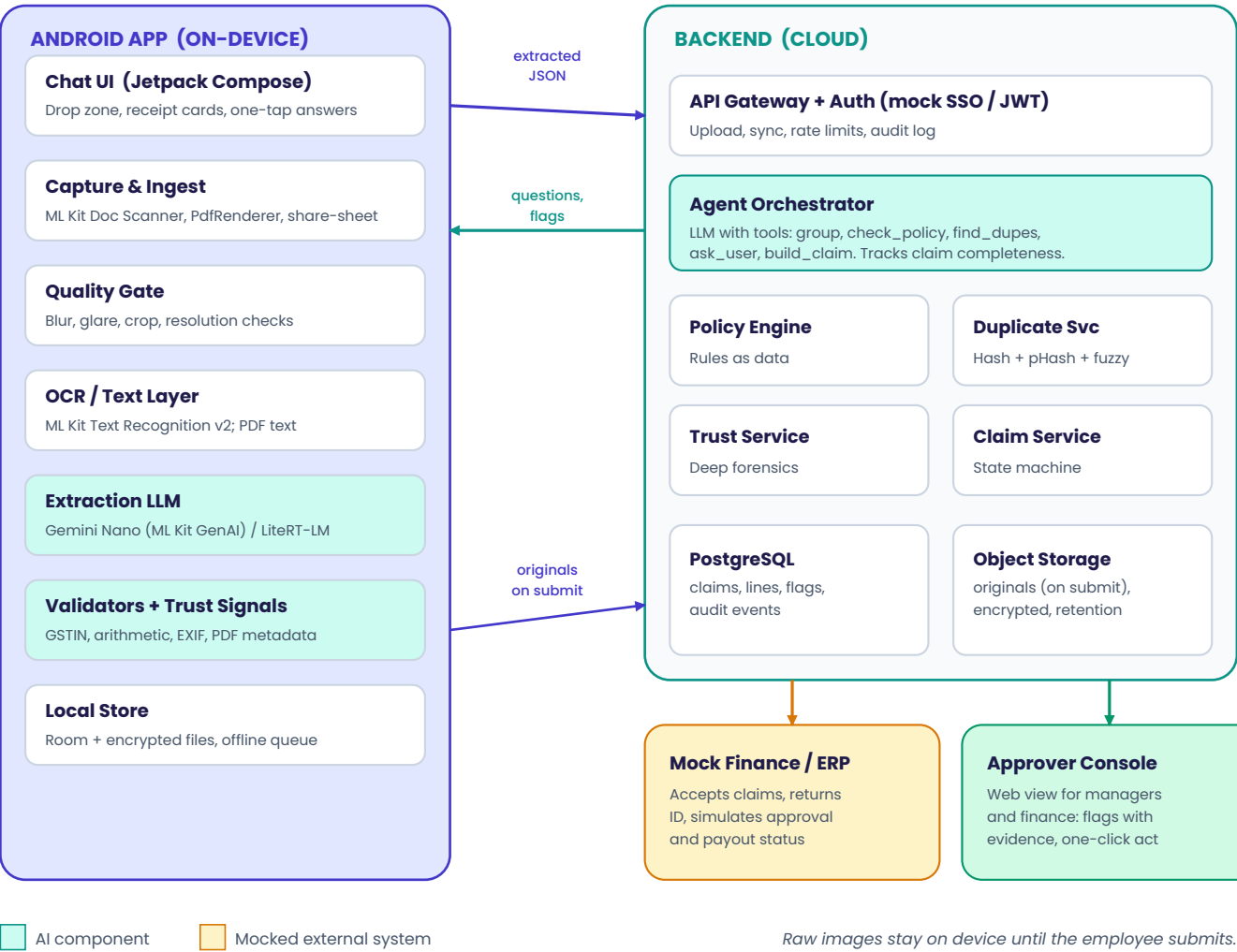


Figure 6. Reading and first-pass checks happen on the phone; the backend owns the claim, policy, duplicates and deep forensics.

Suggested technology choices

Layer	Choice	Reason
Android app	Kotlin, Jetpack Compose, Coroutines/Flow, Hilt, Room, WorkManager	Modern stack; background batch processing survives app kills
Capture & OCR	ML Kit Document Scanner, ML Kit Text Recognition v2, PdfRenderer	Free, on-device, well-supported
On-device LLM	ML Kit GenAI Prompt API (Gemini Nano); LiteRT-LM with a small Gemma model as fallback	Covers capable devices and a broader fallback tier
Backend	Python + FastAPI, PostgreSQL, Redis + RQ, S3-compatible storage (MinIO)	One language for API, forensics and gateway; light on 1 GB VMs (Section 20)
Agent	LLM with tool calling through the LLM gateway (Section 13); tools are backend functions	Provider can change by config; tools keep numbers deterministic
Policy engine	JSON rules evaluated with JsonLogic-style evaluator	Rules editable without code changes
Forensics	Python service: Pillow/OpenCV for ELA and pHash, pypdf for metadata	Mature libraries
Mock finance	Small REST service with simulated approval timeline and webhooks	Demonstrates submission and status end to end
Approver console	React web app (or Compose for web)	Light; reads the same claim API

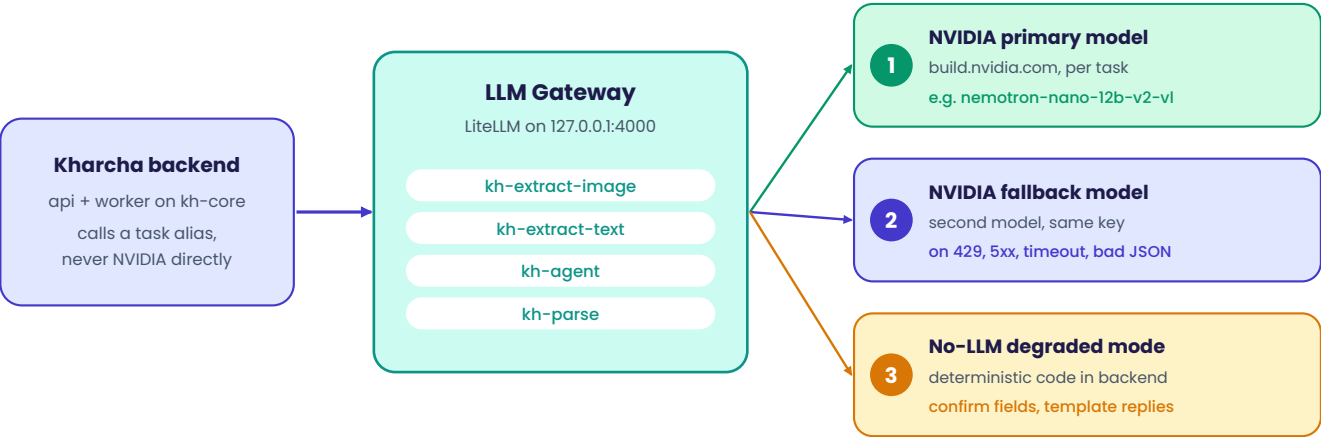
13

NVIDIA HOSTED MODELS

Model provider and LLM gateway

All cloud AI in the prototype runs on **NVIDIA's hosted models** from build.nvidia.com, which are free for prototyping through the NVIDIA Developer Program and use an OpenAI-compatible API. **No model runs on the Oracle VM**; the VM only hosts the backend and the gateway. The backend never calls NVIDIA directly: it asks a small **LLM gateway** (LiteLLM) for a task alias, and the gateway picks a primary NVIDIA model, falls back to a second NVIDIA model, and reports failure so the backend can switch to a no-LLM mode.

LLM GATEWAY: NVIDIA PRIMARY, NVIDIA FALLBACK, THEN NO-LLM MODE



Every call: timeout, one retry on 429, JSON-schema check, then the next step. All calls logged to llm_calls. No model runs on the VM.

Figure 7. Task aliases map to NVIDIA models; when both fail, deterministic code keeps the user moving.

Routing by task

Task alias	Used for	Primary (NVIDIA)	Fallback (NVIDIA)	If both fail
kh-extract-image	Tier C phones: receipt image straight to JSON	nemotron-nano-12b-v2-vl	llama-3.1-nemotron-na-no-vl-8b-vl	User confirms total, date, vendor
kh-extract-text	OCR text to receipt JSON	nvidia-nemotron-na-no-9b-v2	mistral-nemotron	User confirms total, date, vendor
kh-agent	Agent turns with tool calling	mistral-nemotron	nvidia-nemotron-nano-9b-v2	Template replies from the question planner
kh-parse	Free-text answers ("dinner with 2 clients")	nvidia-nemotron-na-no-9b-v2	mistral-nemotron	Ask again with chips only

Model ids are those listed on build.nvidia.com when this PRD was written; confirm each on its model page (task H-05) before building.

What to expect from the free endpoints

- **Free for prototyping:** a Developer Program key (it starts with nvapi-) unlocks the hosted endpoints. Production use would need an NVIDIA AI Enterprise licence.
- **Limits vary:** rate limits depend on the model and account and are not guaranteed. A second model helps when a single model is busy; if the limit is account-wide, the no-LLM mode takes over.
- **Language:** the Nemotron vision models list English only, so Hindi or mixed-script receipts go through on-device OCR first and are confirmed by the user when unsure.
- **Speed:** throughput can drop at peak times. Warm the endpoints before the demo and keep a recorded backup run.
- **One key, two users:** the dev runner and the hosted stack on the same VM share the key. Development stays in stub mode unless a task needs live calls.

Gateway rules

- **Timeouts:** 20 s for extraction, 8 s for agent turns, 5 s for parsing. A timeout counts as a failure.
- **Retries:** one retry with backoff on HTTP 429 or 5xx, then the fallback model. After 3 failures a model is cooled down for 5 minutes.
- **Same contract for both models:** the same versioned prompt and JSON schema from contracts/; output must validate or it counts as a failure.
- **Logging:** every call writes model, task, latency, tokens and outcome to `llm_calls`; the metrics slide is built from it.
- **Key on the server only:** `NVIDIA_API_KEY` lives in the hosted `.env` on `kh-core`. The Android app only talks to Kharcha's API.
- **Swappable later:** another provider can be added by gateway config alone; the prototype uses NVIDIA only.

No-LLM degraded mode

If both models fail, nothing blocks the employee. Phones on Tier A or B keep their on-device extraction. Otherwise the backend stores the OCR text, pre-fills total, date and vendor with simple pattern matching, and asks the user to confirm them. The assistant replies from templates driven by the question planner, and the app shows a small "assistant in basic mode" banner while `/vi/healthz` reports `llm: degraded`.

Data note
Only synthetic or public receipts are sent to NVIDIA during the prototype. A production rollout would move to an enterprise agreement with data-processing terms or to self-hosted models.

14

CORE ENTITIES

Data model

Entity	Key fields	Notes
Employee	id, name, grade, cost_centre, manager_id, home_city	Synthetic seed data
Policy	id, version, rules[] (JSON), effective_from	Versioned; each check stores the version used
Claim	id, employee_id, title, trip_id?, status, total_paise, created_at, submitted_at	State machine in Section 15
ExpenseLine	id, claim_id, document_id, category, date, vendor_id, amount_paise, tax, purpose, attendees[]	One line per receipt (or split receipt)
Document	id, sha256, phash, mime, pages, capture_source, storage_uri, trust_score	Original stored only after submit
ExtractedField	document_id, name, value, confidence, bbox, source (ocr llm user inferred)	Full provenance for every value
Entitlement	employee_id, category, period, period_start, period_end, limit_paise, paid_paise, overall_cap_paise?, fetched_at	Copied from mock ERP; ERP is the source of truth
Reservation	id, entitlement_id, claim_id, line_id, amount_paise, status (held released consumed)	Prevents two drafts using the same balance
Flag	id, target (line doc claim), type, severity, rule_id?, evidence, status, resolution_note	Types: policy, entitlement, duplicate, authenticity, missing
Question	id, claim_id, field, prompt, options[], answer, asked_at, answered_at	Feeds the questions-per-claim metric
AuditEvent	id, actor, action, target, before, after, at	Append-only

15

CONTRACTS

APIs and claim lifecycle

CLAIM LIFECYCLE

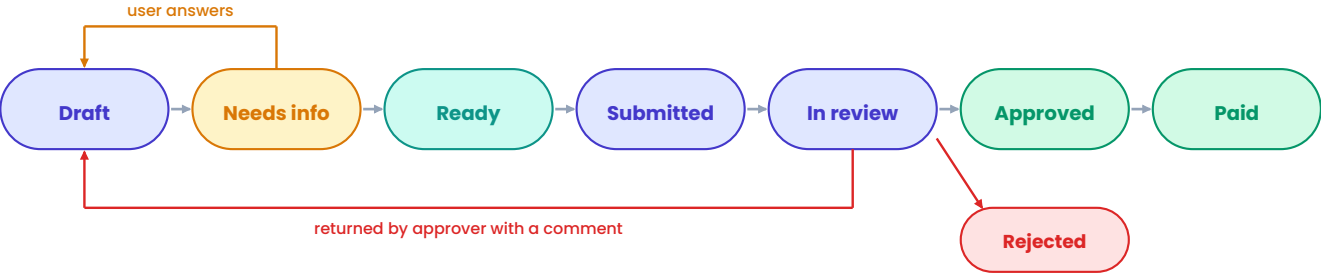


Figure 8. Claim states. Only "Ready" claims can be submitted.

Method & path	Purpose
POST /v1/documents	Register a document (hash, metadata, on-device extraction JSON). Image upload deferred until submit.
POST /v1/documents/{id}/extract	Server-side extraction for Tier C devices (redacted text) or re-extraction.
GET /v1/claims/draft	Current draft claims with completeness state and open questions.
POST /v1/agent/messages	Send a user message or answer; returns agent reply, updated claim and next questions.
POST /v1/claims/{id}/validate	Run policy, duplicate and trust checks; returns flags with evidence.
POST /v1/claims/{id}/submit	Upload originals, freeze the claim, forward to mock finance.
GET /v1/claims/{id}	Full claim with lines, flags, audit trail and status.
POST /v1/approvals/{id}	Approver action: approve, return (with comment) or reject.
GET /v1/entitlements	Employee's balances by category with paid, pending, reserved and available amounts.
GET /mock-erp/employees/{id}/entitlements	Mock ERP source of per-employee limits, periods and paid amounts.
POST /mock-erp/reimbursements	Mock finance intake; returns reference, updates paid amounts and drives status webhooks.

Agent tools

The agent never computes money or policy itself. It calls deterministic tools and narrates the results: `group_expenses`, `check_policy`, `find_duplicates`, `get_trust_report`, `get_entitlements`, `ask_user`, `update_line`, `build_claim` and `submit_claim`. Tool calls and results are logged for audit and evaluation.

16

QUALITY BAR

Non-functional requirements

Area	Requirement
Performance	On-device extraction p50 < 3 s, p95 < 6 s per receipt on a mid-range phone; batch of 10 < 30 s; UI stays at 60 fps during processing.
Offline	Capture, extraction, policy pre-check and drafting work fully offline; sync when online via WorkManager.
Privacy	Raw images stay on device until submit. Card numbers and phone numbers are masked before any upload. Data minimisation in line with India's DPDP Act, 2023.
Security	Encrypted local storage (Android Keystore-backed keys), TLS 1.2+, short-lived JWTs, signed upload URLs, role-based access for approvers.
Reliability	Idempotent uploads keyed by document hash; retries with backoff; no claim lost on app kill.
Explainability	Every auto-filled value links to its source region; every flag lists the signals and rule behind it.
Auditability	Append-only audit log of extraction, edits, answers, overrides and approvals with policy version.
Accessibility	TalkBack labels, dynamic type, minimum 48 dp targets, colour-independent status (icons and text, not colour alone).
Footprint	App < 40 MB excluding optional model; model downloaded on Wi-Fi with user consent.

17

HACKATHON EXECUTION

Build plan and demo

Phase	Scope	Exit criteria
0. Foundations	Repo, CI, docker-compose backend, synthetic data generator, golden and tampered sets	300 labelled docs; CI runs eval script
1. Read	Capture, quality gate, OCR, on-device extraction with schema, device tiering	≥ 90% key-field accuracy on golden set
2. Check	Validators, policy engine, entitlement service with mock ERP limits, duplicate service, trust score v1	Planted duplicates, breaches and exhausted limits flagged
3. Converse	Agent orchestrator with tools, grouping, question policy, chat UI	10-receipt scenario reaches Ready with ≤ 2 questions
4. Submit	Mock finance API, status updates, approver console	Submit, return and approve paths work end to end
5. Polish	Forensics v2, metrics dashboard, demo script, README and architecture notes	Demo under 3 minutes, metrics slide from real eval run

A day-by-day breakdown of these phases is in Section 28.

Demo storyline (3 to 4 minutes)

- Priya drops 12 files from a Pune trip, including a crumpled meal bill, two cab PDFs and a broadband bill. Airplane mode is on to show on-device reading.
- Within seconds, cards fill in. Kharcha groups six items into "Pune client visit" and keeps broadband as a separate claim, showing ₹2,001 of her ₹15,000 broadband limit left after it.
- Two things need her: the hotel is over the city cap, and a cab receipt was already claimed last month. She answers the hotel question with one tap and removes the duplicate.
- A hotel PDF that was edited in a PDF tool is flagged amber with the reason shown. She adds a note.
- Switch to Arjun: his broadband limit is already used up, so Kharcha says so the moment the bill is read and offers to keep it for records instead.
- She taps Submit. The mock finance system returns a reference; Meera approves from the console; the status updates to Approved in Priya's thread.
- Close with the metrics slide: time to ready, questions per claim and accuracy from the evaluation run.

Repository layout

kharcha/ android/ backend/ forensics/ mock-erp/ console/ data/ eval/ docs/	Compose app, ML Kit + LiteRT-LM integration, benchmark module API, agent orchestrator, policy engine, claim service Python trust service (ELA, pHash, PDF metadata) Mock finance API with simulated approvals Approver web console Synthetic generator, golden + tampered sets, labels Evaluation scripts, results history PRD, architecture, demo script
--	--

18

WHAT COULD GO WRONG

Risks and mitigations

Risk	Impact	Mitigation
On-device LLM unavailable on demo device	Core feature does not run	Three-tier fallback; test on target device early; keep cloud path ready
Model invents a field value	Wrong amounts reimbursed	Deterministic validators; values must appear in OCR text; low confidence becomes a question
False fraud flags	Employee trust damaged	Tune on tampered set; amber not red by default; explainable reasons; no auto-reject
Thermal throttling on batches	Slow, hot phone	Thermal checks per document; pause and resume; defer to charging
Handwritten or faded bills	Low extraction accuracy	Retake prompts; crop-and-confirm questions; manual entry as last resort
Privacy or data leakage	Compliance breach	Synthetic data only; on-device by default; PII masking; no real company data in personal accounts
Free-tier limits change (Oracle, NVIDIA)	Demo breaks or slows	NVIDIA fallback model; no-LLM degraded mode; dev in stub mode; check limits the week before the demo
Stale or wrong ERP balances	Over- or under-reimbursement	Server re-check at submit; show "as of" time; ERP stays source of truth
Scope creep in a time-boxed build	Unfinished demo	P0 list is the demo; P1 and P2 only after Phase 4 passes

19

NEXT STEPS

Open questions and future scope

Open questions

- Which demo devices are available, and do any support ML Kit GenAI on-device features?
- Should the agent conversation run on device as well for Tier A, or always on the backend?
- What is the minimum evidence needed for an approver to override a red trust score?
- Should per-diem be modelled as an allowance (no receipts) or as capped actuals?

Future scope

- Corporate card feed matching and automatic receipt chasing for card transactions without a bill.
- Mileage claims from trip GPS traces with privacy controls.
- Real ERP connectors and payout integration once the prototype is validated.
- Org-level analytics: spend by category, policy breach hotspots, leakage recovered.
- Multi-currency travel and per-country tax rules.

Data and compliance note for the build

All development uses synthetic, anonymised or publicly available receipts. No real employee receipts, company policy documents or credentials are used in personal accounts or repositories. Public datasets are used only under their published licences, and the finance system is a mock built for this prototype.

PART B

Development guide

Everything needed to build, deploy and run Kharcha: technology decisions, Android and backend class design, the database schema, step-by-step Oracle Cloud setup, Docker and model installation, provider keys, CI/CD, a day-by-day plan, and a task board that Claude Code can execute.

- 20 Engineering setup

22 Backend design

24 Oracle Cloud setup

26 Model provider setup

28 Day-by-day plan

30 Two-workspace development
- 21 Android app design

23 Database design

25 Installing the stack

27 CI/CD and operations

29 Building with Claude Code

A Appendix: task board

20

DECISIONS AND CONVENTIONS

Engineering setup and repository

Final technology decisions

Area	Decision	Why
Android	Kotlin, Jetpack Compose, Hilt, Room, WorkManager, Retrofit + kotlinx.serialization, Coroutines/Flow	Standard modern stack; batch work survives process death
On-device AI	ML Kit Document Scanner, ML Kit Text Recognition v2, ML Kit GenAI Prompt API (Gemini Nano), LiteRT-LM	Three device tiers (Section 10)
Backend language	Python 3.12 + FastAPI, SQLAlchemy 2, Alembic, Pydantic v2	One language for API, forensics (OpenCV, pypdf, imagehash) and LiteLLM; small memory footprint next to Claude Code on one VM
Jobs	Redis 7 + RQ workers	Simple queue; easy to inspect
Data	PostgreSQL 16, MinIO (S3 API)	Relational claims data; originals in object storage
LLM	NVIDIA hosted models (build.nvidia.com) only, via a LiteLLM gateway with task aliases and NVIDIA fallback models	Free for prototyping; no model on the VM (Section 13)
Hosting	One Oracle VM (kh-core) for development and hosting; Docker Compose stack built on the VM	Fits in 12 GB without a local model (Sections 24 to 25)
Edge	Caddy 2 with automatic HTTPS on kh-core	TLS with two lines of config
Console	React + Vite, built to static files served by Caddy	No server process needed
CI/CD	GitHub Actions for tests; deploy is make deploy on the VM	Simple and visible; no image registry needed
Dev tool	Claude Code terminal CLI on both machines	Same workflow on laptop and VM (Section 30)

Repository layout

```
kharcha/
  android/                                Android Studio project
    app/src/main/java/ai/kharcha/         packages listed in Section 21
  benchmark/                             macrobenchmark + extraction latency harness
  backend/
    app/
      api/                               FastAPI routers (documents, claims, agent, entitlements, approvals, auth)
      core/                             settings, security, logging, errors
      db/                               SQLAlchemy base, session, Alembic migrations
      models/                           ORM models (Section 23)
      schemas/                          Pydantic request/response + receipt JSON schema
      services/                         claim, policy, entitlement, duplicate, trust, extraction, erp client
      agent/                            orchestrator, tool registry, question planner, prompts/
      llm/                              gateway client, prompt templates, schema validation
      workers/                          RQ jobs
    tests/                              pytest (unit + API)
  Dockerfile
  forensics/                             FastAPI internal service: ELA, pHash, PDF metadata (Dockerfile)
  mock-erp/                              FastAPI mock finance: employees, entitlements, intake, status (Dockerfile)
  console/                               React approver console (built to console/dist)
  contracts/                             openapi.yaml, receipt.schema.json, test-vectors/, prompts/ (shared, Section 30)
  scripts/                               laptop helpers: mock-api.sh, tunnel.sh, app-dev-connect.sh
  gateway/                               litellm.config.yaml, Dockerfile
  deploy/
    server/docker-compose.yml  server/Caddyfile  server/initdb/
    scripts/ (bootstrap_server, deploy, backup, llm_smoke)
  data/                                  seed/, generator, golden + tampered sets, labels.csv
  eval/                                 run_eval.py, results/
  .github/workflows/               android.yml backend.yml eval.yml deploy.yml
  .env.example                     hosted stack variables .env.dev.example dev runner variables
```

Conventions

- Trunk-based: short-lived branches, pull requests with CI green before merge, conventional commit messages.
- Secrets never committed. `.env.example` lists variables; real values live in each VM's `.env` (`chmod 600`) and in GitHub Actions secrets.
- Money is always an integer in paise; dates are ISO 8601; IDs are UUIDv7 strings (time-ordered).
- Prompts and JSON schemas are versioned files; every extraction stores the prompt version it used.

Developer machine prerequisites

Tool	Use
Laptop: Android SDK (Studio or command-line tools), JDK 17, Claude Code CLI	Android app; a physical test phone for ML Kit and on-device model tests
On kh-core: Docker, uv, tmux, Claude Code	Installed by <code>deploy/scripts/bootstrap_server.sh</code>
Git, GitHub account, SSH key pair	Code hosting and SSH access to Oracle VMs
Node.js LTS	Build the approver console
Oracle Cloud account (Always Free), NVIDIA Developer key	The one VM and the model provider (Sections 24 to 26)

21

PACKAGES AND CLASSES

Android app design

The app follows a three-layer structure: **ui** (Compose screens and ViewModels), **domain** (use cases and pure logic such as validators) and **data** (Room, network, files). AI components sit in their own **ai** package behind interfaces so the device tier can be switched at runtime.

```
ai.kharcha
  KharchaApp.kt                @HiltAndroidApp
  MainActivity.kt              single activity, KharchaNavHost
  ui/
    chat/                     ChatScreen, ChatViewModel, ChatUiState, ReceiptCard, QuestionChips
    review/                   ReviewScreen, ReviewViewModel, TripGroupCard, FlagBanner, EntitlementStrip
    submit/                   SubmitScreen, SubmitViewModel
    capture/                  CaptureCoordinator (scanner, picker, share intent)
    theme/                    KharchaTheme, tokens
  domain/
    model/                   Receipt, ExtractedField<T>, ExpenseLine, ClaimDraft, Flag, Question, Entitlement
    usecase/                  IngestDocumentsUseCase, ExtractReceiptUseCase, GroupExpensesUseCase,
                              PreviewPolicyUseCase, AnswerQuestionUseCase, SubmitClaimUseCase
    validate/                 GstinValidator, TaxMathValidator, DateSanityValidator, AmountNormalizer
  ai/
    tier/                     DeviceTierResolver, DeviceTier, ThermalGuard
    ocr/                      OcrEngine (ML Kit), PdfTextExtractor, QualityGate
    extract/                  ReceiptExtractor (interface), GeminiNanoExtractor, LiteRtLmExtractor,
                              ServerExtractor, ExtractionPromptBuilder, ReceiptJsonParser
    trust/                    TrustSignalCollector, ExifInspector, PdfMetadataInspector, CaptureSourceDetector
  data/
    db/                      KharchaDatabase, entities, DAOs, Converters
    net/                      KharchaApi (Retrofit), AuthInterceptor, dto/
    repo/                     DocumentRepository, ClaimRepository, EntitlementRepository, PolicyRepository
    files/                    SecureFileStore (Keystore-backed AES-GCM)
    work/                     ExtractionWorker, SyncWorker, EntitlementRefreshWorker
  di/                         AppModule, AiModule, DataModule, NetworkModule
```

Key classes

Class	Responsibility
CaptureCoordinator	Launches the ML Kit Document Scanner, the multi-select picker and handles ACTION_SEND / SEND_MULTIPLE intents; returns content URIs.
IngestDocumentsUseCase	Copies each file into SecureFileStore, computes SHA-256, renders PDF pages with PdfRenderer, creates DocumentEntity rows and enqueues ExtractionWorker.
QualityGate	Laplacian variance for blur, highlight ratio for glare, minimum short edge for resolution; returns Pass or Retake(reason).
OcrEngine	Wraps ML Kit TextRecognizer (Latin + Devanagari); returns text blocks with bounding boxes.
DeviceTierResolver	Checks GenAI feature status, total RAM (ActivityManager.MemoryInfo) and whether a LiteRT-LM model is downloaded; returns Tier A, B or C.
ThermalGuard	Listens to PowerManager thermal status; exposes canRunHeavyWork() used by ExtractionWorker before every document.
ReceiptExtractor	Interface: suspend fun extract(input: ExtractionInput): ExtractionResult. Implementations per tier; the worker picks one via the resolver.
ExtractionPromptBuilder	Builds the prompt from OCR text, the receipt JSON schema and two short examples; versioned (prompt_version).
ReceiptJsonParser	Parses model output with kotlinx.serialization, validates required fields, and on failure retries once with the error message.
GstinValidator / TaxMathValidator	Deterministic checks: 15-character GSTIN pattern, state code and checksum; CGST + SGST or IGST matching rate and total.
TrustSignalCollector	Runs ExifInspector, PdfMetadataInspector and CaptureSourceDetector and returns signals with weights for the trust score.
PreviewPolicyUseCase	Evaluates cached policy rules and entitlement balances locally for instant feedback; the server result is authoritative.
ExtractionWorker	CoroutineWorker processing the queue one document at a time; respects ThermalGuard; writes results to Room; triggers SyncWorker.
SyncWorker	Pushes document metadata and extraction JSON to the API (idempotent by SHA-256); pulls questions, flags and claim state.
ChatViewModel	Single source of truth for the thread: merges Room flows of receipts, questions and claim state into ChatUiState.

Core interfaces (sketch)

```
interface ReceiptExtractor {
    val tier: DeviceTier
    suspend fun isReady(): Boolean
    suspend fun extract(input: ExtractionInput): ExtractionResult
}

data class ExtractionInput(val docId: String, val ocrText: String, val pageBitmap: Bitmap?)
sealed interface ExtractionResult {
    data class Success(val receipt: Receipt, val promptVersion: String) : ExtractionResult
    data class LowConfidence(val receipt: Receipt, val fields: List<String>) : ExtractionResult
    data class Failed(val reason: String) : ExtractionResult
}

class DeviceTierResolver @Inject constructor(
    private val genAi: GenAiStatusChecker, private val memory: MemoryInfoProvider,
    private val liteRt: LiteRtModelStore, private val thermal: ThermalGuard) {
    suspend fun resolve(): DeviceTier = when {
        !thermal.canRunHeavyWork()                -> DeviceTier.C
        genAi.isAvailable()                        -> DeviceTier.A
        memory.totalGb() >= 6 && liteRt.isReady()   -> DeviceTier.B
        else                                       -> DeviceTier.C
    }
}
```

Room tables on the device

Entity	Purpose
DocumentEntity	Local file reference, SHA-256, capture source, quality result, extraction status, trust score
ExtractedFieldEntity	Field name, value, confidence, bounding box, source (ocr, llm, user, inferred)
ExpenseLineEntity / ClaimDraftEntity	Draft lines and grouping before sync; server IDs once synced
QuestionEntity / FlagEntity	Open questions and flags pulled from the server, shown in the thread
EntitlementEntity	Cached balances with fetched_at for offline display
OutboxEntity	Pending API calls for offline-first sync (idempotency key, payload, attempts)

Gradle dependencies (use latest stable versions)

Compose BOM, Material 3, Navigation Compose, Hilt (+ hilt-work), Room (+ KSP), WorkManager, Retrofit, OkHttp, kotlinx.serialization, Coil, androidx.exifinterface, ML Kit Text Recognition v2 (Latin and Devanagari), Play Services ML Kit Document Scanner, ML Kit GenAI Prompt API, LiteRT-LM Android library, Tink (Keystore-backed AES-GCM). Check each library’s release notes for the current artifact name and version, since the GenAI and LiteRT-LM artifacts are still evolving.

22

SERVICES AND CLASSES

Backend design

Container	Host	Role	Memory cap
caddy	kh-core	TLS on :443, reverse proxy to api, serves console static files	64 MB
api	kh-core	FastAPI (uvicorn, 1 worker): auth, documents, claims, agent, entitlements, approvals	384 MB
mock-erp	kh-core	FastAPI mock finance: employees, entitlements, reimbursement intake, status timeline	256 MB
postgres	kh-core	Claims database	1.5 GB
redis	kh-core	RQ job queue, rate-limit counters, circuit-breaker state	256 MB
minio	kh-core	Original documents after submit	512 MB
worker	kh-core	RQ workers: extraction, forensics calls, agent turns, ERP submission	768 MB
forensics	kh-core	Internal FastAPI: ELA, perceptual hash, PDF metadata, recapture heuristics	768 MB
litellm	kh-core	LLM gateway to NVIDIA on 127.0.0.1:4000 with task aliases and fallback models	512 MB
dev runner	kh-core (host)	make dev-api / dev-worker on 127.0.0.1:8001 while developing	~600 MB

Hosted stack about 5 GB, dev runner 0.6 GB and Claude Code about 2 GB, so roughly 8 GB of 12 GB in use, leaving headroom within 12 GB. Adjust caps after measuring with docker stats.

Key classes

Module	Class / function	Responsibility
core	Settings	pydantic-settings; reads .env; one object injected everywhere
api	documents, claims, agent, entitlements, approvals routers	Thin HTTP layer; validates with Pydantic; calls services; never touches LLMs directly
services	ClaimService	Create and update drafts, totals, grouping results; applies ClaimStateMachine transitions
services	ClaimStateMachine	Allowed transitions (Section 15); raises on illegal moves; writes AuditEvent
services	PolicyEngine	Loads active policy version; evaluates JsonLogic rules per line; returns Flag objects with rule_id and remedies
services	EntitlementService	Fetches from ErpClient, computes available = limit - paid - pending - reserved; creates and releases reservations inside a transaction with row locks
services	DuplicateService	Exact SHA-256 match, pHash Hamming distance ≤ 6 , and fuzzy key (vendor, date, amount $\pm 1\%$) across user and organisation
services	TrustService	Combines device signals with forensics results into a 0 to 100 score with top reasons
services	ExtractionService	Tier C extraction via LLMGateway; schema validation; stores ExtractedField rows
services	ErpClient	HTTP client for mock-erp; retries; maps ERP errors to domain errors
agent	AgentOrchestrator	Runs one agent turn: builds context, calls kh-agent with tools, executes tool calls, returns reply and updated claim
agent	ToolRegistry + tools	group_expenses, check_policy, get_entitlements, find_duplicates, get_trust_report, ask_user, update_line, build_claim, submit_claim
agent	QuestionPlanner	Deterministic ask() rule from Section 7; ranks and batches questions; the LLM only words them
llm	LLMGateway	OpenAI client pointed at LiteLLM; task alias in, validated JSON out; writes llm_calls rows
workers	extract_document, run_forensics, agent_turn, submit_to_erp, refresh_entitlements	RQ jobs with timeouts and retry policies

Agent turn (sketch)

```
class AgentOrchestrator:
    MAX_STEPS = 6

    async def run_turn(self, claim_id: str, user_msg: str | None) -> AgentReply:
        ctx = await self.context.build(claim_id)          # lines, flags, open questions, balances
        planned = self.planner.next_questions(ctx)        # deterministic, max 3
        messages = self.prompts.render("agent_v3", ctx=ctx, planned=planned, user=user_msg)
        for _ in range(self.MAX_STEPS):
            resp = await self.llm.chat("kh-agent", messages, tools=self.tools.schemas())
            if not resp.tool_calls:
                return AgentReply(text=resp.text, claim=await self.claims.view(claim_id))
            for call in resp.tool_calls:
                result = await self.tools.execute(call.name, call.args, claim_id=claim_id)
                messages += [resp.as_message(), tool_result(call.id, result)]
        return AgentReply(text=self.prompts.fallback(planned), claim=await self.claims.view(claim_id))
```

Entitlement reservation (sketch)

```
async def reserve(self, session, line: ExpenseLine) -> Reservation:
    ent = await session.scalar(
        select(Entitlement).where(Entitlement.employee_id == line.employee_id,
                                Entitlement.category == line.category,
                                Entitlement.period_start <= line.expense_date,
                                Entitlement.period_end >= line.expense_date)
        .with_for_update() # row lock prevents double use
    )
    if ent is None:
        raise NotEntitled(line.category)
    available = ent.limit_paise - ent.paid_paise - await self.pending(session, ent) \
        - await self.reserved(session, ent, exclude_line=line.id)
    covered = max(0, min(line.amount_paise, available))
    return Reservation(entitlement_id=ent.id, line_id=line.id, amount_paise=covered, status="held")
```

Mock ERP contract

Endpoint	Behaviour
GET /employees/{id}	Synthetic employee with grade, cost centre, manager
GET /employees/{id}/entitlements	Per-category limits, periods and paid amounts; values differ by grade and per employee seed file
POST /reimbursements	Validates claim schema, returns ERP reference, schedules status changes (in review, approved, paid) on a timer
GET /reimbursements/{ref}	Current status; paid amounts update the employee's entitlement
POST /admin/scenario	Demo helper: set an employee's broadband as fully used, force a rejection, add latency

23

POSTGRESQL SCHEMA

Database design

All tables use UUID primary keys and created_at / updated_at timestamps (omitted below for brevity). Money is stored as BIGINT paise. Enumerations are PostgreSQL enum types so invalid states cannot be written. Migrations are managed with Alembic.

Table	Relations	Main indexes
employees	manager_id to employees	email unique
policies	referenced by flags.policy_version	(version) unique, active partial index
entitlements	employee_id	(employee_id, category, period_start) unique
claims	employee_id; approver_id to employees	(employee_id, status)
documents	employee_id; claim_id nullable	sha256 unique per employee; phash btree; fuzzy_key
expense_lines	claim_id, document_id	(claim_id), (category, expense_date)
reservations	entitlement_id, line_id	(entitlement_id, status)
extracted_fields	document_id	(document_id, name)
flags	claim_id, line_id or document_id	(claim_id, status)
questions	claim_id, line_id nullable	(claim_id) where answered_at is null

Table	Relations	Main indexes
agent_messages	claim_id	(claim_id, created_at)
llm_calls	claim_id, document_id nullable	(task, created_at), (provider, success)
audit_events	actor_id, target	(target_type, target_id, at)

```
CREATE TYPE claim_status AS ENUM ('draft','needs_info','ready','submitted','in_review',
                                   'approved','rejected','returned','paid');
CREATE TYPE expense_category AS ENUM ('flight','rail','cab','hotel','meal','fuel','toll',
                                       'telecom','broadband','other');
CREATE TYPE flag_type AS ENUM ('policy','entitlement','duplicate','authenticity','missing');
CREATE TYPE severity AS ENUM ('green','amber','red');
CREATE TYPE period_kind AS ENUM ('month','quarter','fin_year','per_trip');

CREATE TABLE employees (
  id uuid PRIMARY KEY, email text UNIQUE NOT NULL, name text NOT NULL,
  grade text NOT NULL, cost_centre text, home_city text,
  manager_id uuid REFERENCES employees(id), role text NOT NULL DEFAULT 'employee');

CREATE TABLE policies (
  id uuid PRIMARY KEY, version int UNIQUE NOT NULL, rules jsonb NOT NULL,
  effective_from date NOT NULL, active boolean NOT NULL DEFAULT false);

CREATE TABLE entitlements (
  id uuid PRIMARY KEY, employee_id uuid NOT NULL REFERENCES employees(id),
  category expense_category, overall boolean NOT NULL DEFAULT false,
  period period_kind NOT NULL, period_start date NOT NULL, period_end date NOT NULL,
  limit_paise bigint NOT NULL CHECK (limit_paise >= 0),
  paid_paise bigint NOT NULL DEFAULT 0, fetched_at timestampz NOT NULL,
  UNIQUE (employee_id, category, period_start));

CREATE TABLE claims (
  id uuid PRIMARY KEY, employee_id uuid NOT NULL REFERENCES employees(id),
  title text, trip_start date, trip_end date, city text,
  status claim_status NOT NULL DEFAULT 'draft',
  total_paise bigint NOT NULL DEFAULT 0, approver_id uuid REFERENCES employees(id),
  erp_ref text, submitted_at timestampz, decided_at timestampz);
CREATE INDEX ON claims (employee_id, status);

CREATE TABLE documents (
  id uuid PRIMARY KEY, employee_id uuid NOT NULL REFERENCES employees(id),
  claim_id uuid REFERENCES claims(id), sha256 char(64) NOT NULL, phash bigint,
  fuzzy_key text, mime text NOT NULL, pages int NOT NULL DEFAULT 1,
  capture_source text NOT NULL, storage_uri text, trust_score smallint,
  device_tier char(1), prompt_version text,
  UNIQUE (employee_id, sha256));
CREATE INDEX ON documents (fuzzy_key);
CREATE INDEX ON documents (phash);
```

```

CREATE TABLE expense_lines (
  id uuid PRIMARY KEY, claim_id uuid NOT NULL REFERENCES claims(id) ON DELETE CASCADE,
  document_id uuid REFERENCES documents(id), category expense_category NOT NULL,
  expense_date date NOT NULL, vendor text, city text,
  amount_paise bigint NOT NULL, claimable_paise bigint NOT NULL,
  cgst_paise bigint, sgst_paise bigint, igst_paise bigint, gstin text,
  purpose text, attendees jsonb NOT NULL DEFAULT '[]', excluded_reason text);
CREATE INDEX ON expense_lines (claim_id);

CREATE TABLE reservations (
  id uuid PRIMARY KEY, entitlement_id uuid NOT NULL REFERENCES entitlements(id),
  line_id uuid NOT NULL REFERENCES expense_lines(id) ON DELETE CASCADE,
  amount_paise bigint NOT NULL,
  status text NOT NULL CHECK (status IN ('held','released','consumed')));
CREATE INDEX ON reservations (entitlement_id, status);

CREATE TABLE extracted_fields (
  id uuid PRIMARY KEY, document_id uuid NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
  name text NOT NULL, value jsonb NOT NULL, confidence real NOT NULL,
  bbox int[4], source text NOT NULL CHECK (source IN ('ocr','llm','user','inferred')));

CREATE TABLE flags (
  id uuid PRIMARY KEY, claim_id uuid NOT NULL REFERENCES claims(id) ON DELETE CASCADE,
  line_id uuid REFERENCES expense_lines(id), document_id uuid REFERENCES documents(id),
  type flag_type NOT NULL, severity severity NOT NULL, rule_id text,
  policy_version int, message text NOT NULL, evidence jsonb NOT NULL DEFAULT '{}',
  status text NOT NULL DEFAULT 'open', resolution_note text);

CREATE TABLE questions (
  id uuid PRIMARY KEY, claim_id uuid NOT NULL REFERENCES claims(id) ON DELETE CASCADE,
  line_id uuid REFERENCES expense_lines(id), field text NOT NULL, prompt text NOT NULL,
  options jsonb NOT NULL DEFAULT '[]', answer jsonb,
  asked_at timestampz NOT NULL DEFAULT now(), answered_at timestampz);

CREATE TABLE agent_messages (
  id uuid PRIMARY KEY, claim_id uuid NOT NULL REFERENCES claims(id) ON DELETE CASCADE,
  role text NOT NULL, content jsonb NOT NULL, created_at timestampz NOT NULL DEFAULT now());

CREATE TABLE llm_calls (
  id uuid PRIMARY KEY, task text NOT NULL, provider text NOT NULL, model text NOT NULL,
  claim_id uuid, document_id uuid, latency_ms int, tokens_in int, tokens_out int,
  success boolean NOT NULL, failure text, created_at timestampz NOT NULL DEFAULT now());

CREATE TABLE audit_events (
  id uuid PRIMARY KEY, actor_id uuid, action text NOT NULL, target_type text NOT NULL,
  target_id uuid NOT NULL, before jsonb, after jsonb, at timestampz NOT NULL DEFAULT now());

```

24

ONE VM FOR DEVELOPMENT AND HOSTING

Oracle Cloud setup

ONE ORACLE ALWAYS FREE VM: DEVELOPMENT + HOSTING



Dev and hosted share Postgres, Redis, MinIO and the gateway on 127.0.0.1 but use separate databases, Redis DBs, buckets and queues.

Figure 9. kh-core runs the hosted stack and the development environment side by side; only Caddy is public.

Check the free allowance first

Oracle reduced the Always Free Ampere A1 allowance to 2 OCPUs and 12 GB in total during 2026. Size kh-core to exactly 2 OCPU / 12 GB and confirm the current limits on Oracle's Always Free resources page. The two micro VMs in the free tier are not needed for this design.

Step 1. Network

In the OCI Console open **Networking > Virtual Cloud Networks > Start VCN Wizard** and choose **Create VCN with Internet Connectivity** (CIDR 10.0.0.0/16). Place kh-core in the public subnet.

Step 2. Security list ingress rules

Source	Protocol / port	Purpose
Your home IP /32	TCP 22	SSH, Claude Code sessions and the dev tunnel
0.0.0.0/0	TCP 80, 443	Caddy certificate challenge and HTTPS for the app and console

Nothing else is opened. Postgres, Redis, MinIO, the gateway, mock ERP and the dev API listen on 127.0.0.1 only.

Step 3. Create the instance

Name	Shape	Image	Boot volume	Notes
kh-core	VM.Standard.A1.Flex, 2 OCPU, 12 GB	Canonical Ubuntu 24.04 (aarch64)	~100 GB	Public IP; paste your SSH key. If "out of capacity", retry later or another availability domain

Oracle may reclaim Always Free instances it considers idle; check the idle criteria in Oracle's Always Free documentation. A running stack and daily development normally keep kh-core active.

Step 4. Bootstrap the VM

One script installs base tools, swap, Docker with log rotation, the host firewall rules for 80 and 443 (Oracle's Ubuntu images reject other inbound traffic by default), tmux, uv and the Claude Code CLI, and creates the release folder.

```
# from the laptop
scp deploy/scripts/bootstrap_server.sh ubuntu@<kh-core-public-ip>:~
ssh ubuntu@<kh-core-public-ip> 'bash ~/bootstrap_server.sh'
ssh ubuntu@<kh-core-public-ip>          # new login picks up the docker group
claude --version

#!/usr/bin/env bash
# One-time setup of the single Oracle VM kh-core (Ubuntu 24.04 arm64). PRD $24.
# Installs: base tools, 4G swap, Docker + compose, host firewall for 80/443, tmux, uv, Claude Code.
set -euo pipefail

sudo apt update && sudo apt -y upgrade
sudo timedatectl set-timezone Asia/Kolkata
sudo apt -y install git curl jq unzip htop tmux build-essential netfilter-persistent

# swap (safety net; the stacks fit in RAM)
if ! swapon --show | grep -q /swapfile; then
  sudo fallocate -l 4G /swapfile && sudo chmod 600 /swapfile
  sudo mkswap /swapfile && sudo swapon /swapfile
  echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
  echo 'vm.swappiness=10' | sudo tee /etc/sysctl.d/99-swap.conf && sudo sysctl --system
fi

# Docker Engine + compose plugin, with log rotation
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker "$USER"
echo '{"log-driver":"json-file","log-opts":{"max-size":"10m","max-file":"3"}}' \
  | sudo tee /etc/docker/daemon.json && sudo systemctl restart docker

# host firewall: Oracle Ubuntu images reject inbound traffic by default; open web ports only
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 80 -j ACCEPT
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 443 -j ACCEPT
sudo netfilter-persistent save

# Python toolchain and Claude Code (terminal CLI, native installer)
curl -LsSf https://astral.sh/uv/install.sh | sh
curl -fsSL https://claude.ai/install.sh | bash

# release checkout location for the hosted stack
sudo mkdir -p /opt/kharcha && sudo chown "$USER":"$USER" /opt/kharcha

echo "Done. Log out and back in (docker group), then run: claude --version"
```

Docker and firewalls

Ports published by Docker bypass the host INPUT chain, so the compose file publishes every internal service on 127.0.0.1 only. Caddy is the single exception on 80 and 443. The OCI security list remains the outer wall.

Step 5. Domain and HTTPS

Create a free subdomain at a dynamic-DNS provider such as DuckDNS and point it to kh-core's public IP. Caddy obtains and renews the certificate automatically once 80 and 443 are open.

25

TWO CHECKOUTS, ONE VM

Installing the stack

kh-core holds two copies of the repository. The **dev checkout** ~/kharcha is where Claude Code works on feature branches. The **release checkout** /opt/kharcha holds only main or a tag and builds the hosted stack. Keeping them apart means an unfinished change can never reach the demo, and Claude Code is told never to touch /opt/kharcha.

	Dev (Claude Code)	Hosted (demo)
Checkout	~/kharcha, feature branches	/opt/kharcha, main or a tag
Runs as	make dev-api, make dev-worker (host processes)	Docker Compose project kharcha
API	127.0.0.1:8001, reached through the SSH tunnel	Caddy at https://your-domain
Data	kharcha_dev database, Redis DB 1, bucket originals-dev, queues dev-*	kharcha database, Redis DB 0, bucket originals
Env file	~/kharcha/.env.dev (LLM_MODE=stub by default)	/opt/kharcha/.env (LLM_MODE=live)
Who starts it	Claude Code or you	You only: make deploy

Step 1. Checkouts and environment files

```
ssh-keygen -t ed25519 -C kh-core && cat ~/.ssh/id_ed25519.pub      # add to GitHub
git clone git@github.com:<you>/kharcha.git ~/kharcha            # dev checkout
cd ~/kharcha && cp .env.dev.example .env.dev && chmod 600 .env.dev
cp docs/workspaces/CLAUDE.local.BACKEND.md CLAUDE.local.md
git clone git@github.com:<you>/kharcha.git /opt/kharcha          # release checkout
cp /opt/kharcha/.env.example /opt/kharcha/.env && chmod 600 /opt/kharcha/.env
nano /opt/kharcha/.env      # DOMAIN, passwords, NVIDIA_API_KEY (Section 26)
```

Step 2. Hosted stack definition

All images build on the VM (arm64). Official images (Caddy, Postgres, Redis, MinIO) are multi-arch; Kharcha's own Dockerfiles start from python:3.12-slim.

```
# Kharcha hosted stack on the single Oracle VM (kh-core, arm64).
# Run from the RELEASE checkout /opt/kharcha via deploy/scripts/deploy.sh (project name: kharcha).
# Only Caddy is public (80/443). Everything else binds to 127.0.0.1 so the dev runner and
# the SSH tunnel can reach it, but the internet cannot.
name: kharcha
services:
  caddy:
    image: caddy:2
    ports: [ "80:80", "443:443" ]
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - ../../console/dist:/srv/console:ro
      - caddy:/data
    environment:
      DOMAIN: ${DOMAIN}
    depends_on: [ api ]
    mem_limit: 64m
    restart: unless-stopped
  api:
    build: ../../backend
    command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 1
    env_file: ../../.env
    volumes: [ "../../data/seed:/data/seed:ro" ]
    depends_on:
      postgres: { condition: service_healthy }
      redis: { condition: service_started }
    mem_limit: 384m
    restart: unless-stopped
  worker:
    build: ../../backend
    command: rq worker --url redis://redis:6379/0 extract forensics agent erp
    env_file: ../../.env
    depends_on: [ postgres, redis ]
    mem_limit: 768m
    restart: unless-stopped
  forensics:
    build: ../../forensics
    ports: [ "127.0.0.1:8085:8085" ]
    mem_limit: 768m
    restart: unless-stopped
  mock-erp:
    build: ../../mock-erp
    ports: [ "127.0.0.1:8090:8090" ]
    volumes: [ "../../data/seed:/data/seed:ro" ]
    mem_limit: 256m
    restart: unless-stopped
  litellm:
    build: ../../gateway
    env_file: ../../.env
    ports: [ "127.0.0.1:4000:4000" ]
    mem_limit: 512m
    restart: unless-stopped
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: kharcha
      POSTGRES_USER: kharcha
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
```

```

command: postgres -c shared_buffers=256MB -c work_mem=4MB -c max_connections=60
volumes: [ "pg:/var/lib/postgresql/data", "./initdb:/docker-entrypoint-initdb.d:ro" ]
ports: [ "127.0.0.1:5432:5432" ]
healthcheck: { test: ["CMD", "pg_isready", "-U", "kharcha"], interval: 10s }
mem_limit: 1536m
restart: unless-stopped
redis:
image: redis:7-alpine
command: redis-server --appendonly yes --maxmemory 200mb --maxmemory-policy noeviction
volumes: [ "redis:/data" ]
ports: [ "127.0.0.1:6379:6379" ]
mem_limit: 256m
restart: unless-stopped
minio:
image: minio/minio
command: server /data --console-address ":9001"
environment:
  MINIO_ROOT_USER: ${MINIO_ROOT_USER}
  MINIO_ROOT_PASSWORD: ${MINIO_ROOT_PASSWORD}
volumes: [ "minio:/data" ]
ports: [ "127.0.0.1:9000:9000", "127.0.0.1:9001:9001" ]
mem_limit: 512m
restart: unless-stopped
volumes: { pg: {}, redis: {}, minio: {}, caddy: {} }

```

```

# deploy/server/Caddyfile
{$DOMAIN} {
  encode gzip
  handle /v1/* {
    reverse_proxy api:8000
  }
  handle {
    root * /srv/console
    try_files {path} /index.html
    file_server
  }
}

# deploy/server/initdb/10-dev-test.sql
-- Runs once when the Postgres volume is first created.
-- kharcha      : hosted stack (demo)
-- kharcha_dev  : dev runner used while Claude Code develops (make dev-api)
-- kharcha_test : pytest (wiped by tests)
CREATE DATABASE kharcha_dev OWNER kharcha;
CREATE DATABASE kharcha_test OWNER kharcha;

```

Step 3. Start shared services early

Before the backend exists, start the shared infrastructure from the release checkout so development can begin. Add services as their tasks merge.

```

cd ~/kharcha
make infra-up                                # postgres redis minio
make infra-up SVC="postgres redis minio mock-erp forensics litellm" # after T-06 merges
make status                                  # containers + free memory

```

Step 4. Development loop on the VM

```

tmux new -As kharcha
# window 1: Claude Code      window 2: dev API      window 3: dev worker
claude                       make migrate-dev && make dev-api  make dev-worker
curl -s 127.0.0.1:8001/v1/healthz
make backend-test             # kharcha_test DB, LLM stub
make llm-smoke                # one tiny live call per alias (uses NVIDIA quota)

```

Step 5. Deploy the hosted stack (you only)

```
#!/usr/bin/env bash
# Deploy the hosted stack from the RELEASE checkout /opt/kharcha.
# HUMAN-run only (never by Claude Code unless asked).
# Usage: deploy/scripts/deploy.sh [git-ref] (default: main; use a tag like v0.1.0 for the demo)
set -euo pipefail
REF=${1:-main}
REPO_URL=$(git -C "$(dirname "$0")/../../" remote get-url origin)
REL=/opt/kharcha

if [ ! -d "$REL/.git" ]; then
  git clone "$REPO_URL" "$REL"
fi
cd "$REL"
git fetch --tags origin
git checkout --quiet "$REF"
git pull --ff-only origin "$REF" 2>/dev/null || true # no-op for tags
test -f .env || { echo "Missing $REL/.env (copy .env.example and fill it)"; exit 1; }

if [ -d console ] && [ -f console/package.json ]; then
  (cd console && npm ci && npm run build)
fi

DC=(docker compose --env-file .env -f deploy/server/docker-compose.yml)
"${DC[@]}" build
"${DC[@]}" up -d
"${DC[@]}" exec -T api alembic upgrade head
sleep 5
curl -fsS "https://$(grep '^DOMAIN=' .env | cut -d= -f2)/v1/healthz" && echo " <- healthz OK ($REF)"
```

```
# first time and every release
cd ~/kharcha && git tag v0.1.0 && git push --tags
make deploy REF=v0.1.0
docker compose -p kharcha exec api python -m app.scripts.seed \
  --employees /data/seed/employees.json --policy /data/seed/policy_v1.json
```

Step 6. Backups

```
#!/usr/bin/env bash
# Nightly backup of the hosted database. Install with `crontab -e`:
# 15 2 * * * /opt/kharcha/deploy/scripts/backup.sh >> /home/ubuntu/backups/backup.log 2>&1
set -euo pipefail
mkdir -p ~/backups
docker exec kharcha-postgres-1 pg_dump -U kharcha kharcha \
  | gzip > ~/backups/"kharcha_$(date +%F).sql.gz"
find ~/backups -name 'kharcha_*.sql.gz' -mtime +7 -delete
```

KEY, MODELS, GATEWAY

26

NVIDIA model setup

NVIDIA build.nvidia.com is the only model provider. These steps create the key, confirm the models and wire them into the gateway.

Step 1. Get the API key

1. Open build.nvidia.com and sign in or join the free **NVIDIA Developer Program** (email only, no card).
2. Go to **Settings > API Keys** (or **Get API Key** on any model page) and generate a key. It starts with `nvapi-`. Copy it once.
3. On `kh-core`, put it in `/opt/kharcha/.env` as `NVIDIA_API_KEY`. It is never committed and never sent to the phone.

Step 2. Confirm the models (task H-05)

Model id	Role	Check on its build.nvidia.com page
nvidia/nemotron-nano-12b-v2-vl	Receipt images, primary	Free endpoint, image input format in the code sample
nvidia/llama-3.1-nemotron-nano-vl-8b-v1	Receipt images, fallback	Free endpoint, image input format
nvidia/nvidia-nemotron-nano-9b-v2	Text extraction primary, parsing primary, agent fallback	Free endpoint; reasoning toggle if the model card offers one (keep it off for extraction)
mistralai/mistral-nemotron	Agent primary (tool calling), text and parse fallback	Free endpoint, tool-calling support

```
export NVIDIA_API_KEY=nvapi-xxxxxxx
curl -s https://integrate.api.nvidia.com/v1/chat/completions \
-H "Authorization: Bearer $NVIDIA_API_KEY" -H "Content-Type: application/json" \
-d '{"model": "mistralai/mistral-nemotron",
  "messages": [{"role": "user", "content": "Reply with the single word OK"}],
  "max_tokens": 10}' | jq -r '.choices[0].message.content'
```

Step 3. Gateway configuration

```
# Kharcha LLM gateway: NVIDIA hosted models only (build.nvidia.com, OpenAI-compatible).
# Confirm each model id on its build.nvidia.com page before use (H-05). No local model.
# Fallback = a second NVIDIA model.
# If both fail, the backend switches to no-LLM degraded mode (SPEC §6).
model_list:
  - model_name: kh-extract-image
    litellm_params:
      model: nvidia_nim/nvidia/nemotron-nano-12b-v2-v1
      api_key: os.environ/NVIDIA_API_KEY
  - model_name: kh-extract-image-fb
    litellm_params:
      model: nvidia_nim/nvidia/llama-3.1-nemotron-nano-v1-8b-v1
      api_key: os.environ/NVIDIA_API_KEY
  - model_name: kh-extract-text
    litellm_params:
      model: nvidia_nim/nvidia/nvidia-nemotron-nano-9b-v2
      api_key: os.environ/NVIDIA_API_KEY
  - model_name: kh-extract-text-fb
    litellm_params:
      model: nvidia_nim/mistralai/mistral-nemotron
      api_key: os.environ/NVIDIA_API_KEY
  - model_name: kh-agent
    litellm_params:
      model: nvidia_nim/mistralai/mistral-nemotron
      api_key: os.environ/NVIDIA_API_KEY
  - model_name: kh-agent-fb
    litellm_params:
      model: nvidia_nim/nvidia/nvidia-nemotron-nano-9b-v2
      api_key: os.environ/NVIDIA_API_KEY
  - model_name: kh-parse
    litellm_params:
      model: nvidia_nim/nvidia/nvidia-nemotron-nano-9b-v2
      api_key: os.environ/NVIDIA_API_KEY
  - model_name: kh-parse-fb
    litellm_params:
      model: nvidia_nim/mistralai/mistral-nemotron
      api_key: os.environ/NVIDIA_API_KEY

router_settings:
  num_retries: 1
  timeout: 20
  allowed_fails: 3
  cooldown_time: 300
  fallbacks:
    - kh-extract-image: [kh-extract-image-fb]
    - kh-extract-text: [kh-extract-text-fb]
    - kh-agent: [kh-agent-fb]
    - kh-parse: [kh-parse-fb]

general_settings:
  master_key: os.environ/LITELLM_MASTER_KEY

# Not used in the prototype. Another provider can be added later by config only, e.g.:
# - model_name: kh-agent-alt
#   litellm_params: { model: <provider>/<model>, api_key: os.environ/<PROVIDER>_API_KEY }
```

LiteLLM option names change between releases; check its proxy documentation for the version you install.

```
# gateway/Dockerfile
FROM python:3.12-slim
RUN pip install --no-cache-dir "litellm[proxy]"
COPY litellm.config.yaml /app/litellm.config.yaml
EXPOSE 4000
CMD ["litellm", "--config", "/app/litellm.config.yaml", "--host", "0.0.0.0", "--port", "4000"]
```

Step 4. Smoke test

```
# make llm-smoke -> deploy/scripts/llm_smoke.sh
#!/usr/bin/env bash
# Calls each gateway alias once with a tiny prompt. Reads LITELLM_MASTER_KEY from .env.dev.
set -euo pipefail
cd "$(dirname "$0")/../../"
KEY=$(grep '^LITELLM_MASTER_KEY=' .env.dev | cut -d= -f2)
for alias in kh-extract-text kh-agent kh-parse kh-extract-text-fb kh-agent-fb; do
  printf '%-22s ' "$alias"
  body=$(jq -nc --arg m "$alias" \
    '{model:$m, max_tokens:8, messages:[{role:"user", content:"Reply with OK"}]}')
  curl -s -m 30 http://127.0.0.1:4000/v1/chat/completions \
    -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" -d "$body" \
    | jq -r '.choices[0].message.content // .error.message' | head -c 60
  echo
done
```

Environment variables

Variable	File	Example / note
DOMAIN	.env	kharcha.duckdns.org (points to kh-core)
POSTGRES_PASSWORD, DATABASE_URL	.env, .env.dev	hosted: @postgres:5432/kharcha; dev: @127.0.0.1:5432/kharcha_dev
TEST_DATABASE_URL	.env.dev	@127.0.0.1:5432/kharcha_test
REDIS_URL, QUEUE_PREFIX	.env, .env.dev	hosted redis://redis:6379/0; dev redis://127.0.0.1:6379/1 with prefix dev-
MINIO_ROOT_USER/PASSWORD, S3_ENDPOINT, S3_BUCKET	.env, .env.dev	buckets originals and originals-dev
JWT_SECRET	.env, .env.dev	64 random bytes; mock SSO signs demo users
ERP_BASE_URL, FORENSICS_URL	.env, .env.dev	service names in .env; 127.0.0.1 ports in .env.dev
LLM_MODE	.env, .env.dev	live for hosted; stub for dev unless a task needs live calls
LITELLM_URL, LITELLM_MASTER_KEY	.env, .env.dev	http://litellm:4000 or http://127.0.0.1:4000; same key in both
NVIDIA_API_KEY	.env only	Read by the gateway container; the only provider key

27

KEEPING IT RUNNING

CI/CD, testing and operations

Workflow	Trigger	Steps
android.yml	PR, push to main	Gradle cache, lint, unit tests, assembleDebug, upload APK artifact
backend.yml	PR, push to main	ruff + mypy, pytest with Postgres and Redis service containers, Alembic migration check
eval.yml	Manual	Run eval/run_eval.py in stub mode in CI; live NVIDIA runs happen on kh-core with make eval
deploy	You, on kh-core	make deploy REF=v0.x.y builds from /opt/kharcha on the VM, runs migrations and checks healthz (no registry, no deploy workflow)

Testing approach

Level	What	Tools
Unit	Validators, policy rules, entitlement maths, question planner, state machine	JUnit + Turbine (Android), pytest (backend)
Integration	API with real Postgres and Redis; mock ERP; gateway with a stub provider	pytest + testcontainers or CI services
AI evaluation	Field accuracy, category accuracy, tamper recall, questions per claim, latency per provider	eval/run_eval.py, llm_calls table
On device	Extraction latency and memory per tier; thermal behaviour on a 20-receipt batch	Macrobenchmark, Android Studio profiler
End to end	Demo storyline scripted against staging	Maestro or Compose UI tests

Runbook

Symptom	Likely cause	Fix
Site unreachable	Caddy down or ports blocked	make status; check iptables rules (80/443) and the security list
Certificate errors	DNS not pointing to kh-core	Update the DuckDNS record; restart caddy
Extraction stuck "on server"	Worker down or NVIDIA failing	docker logs kharcha-worker-1; check llm_calls failures; make llm-smoke
Many 429 errors	NVIDIA limits reached	Dev back to LLM_MODE=stub; fallback model; degraded mode keeps users moving
kh-core out of memory	Caps too high or too many Claude Code sessions	make status; close idle sessions; lower container caps
Instance stopped by Oracle	Over free limits or idle reclamation	Check the console and Always Free limits; resize to 2 OCPU / 12 GB
Demo broke after a change	Change reached /opt/kharcha	make deploy REF= to roll back

28

FIFTEEN WORKING DAYS

Day-by-day development plan

Each row maps to task cards T-00 to T-32 (Appendix A, docs/TASKS.md); the Android column runs on the laptop and the backend and AI columns on kh-core (Section 30). The plan assumes one developer working full days. If the timeline is shorter, keep every P0 item and drop P1 and P2 items in the order listed in Section 6.

Day	Android	Backend and infrastructure	AI and evaluation	Done when
1	Laptop setup; project skeleton, Compose theme, navigation, Hilt	Oracle VM, bootstrap, Claude Code, both checkouts, infra-up, contract freeze	Synthetic receipt generator v0	Mock API serves contract; ssh kh-core works
2	Capture: scanner, picker, share intent	Postgres schema + Alembic, seed data, mock ERP endpoints	Generate 200 synthetic + tampered variants	Mock ERP returns entitlements
3	Ingest, SecureFileStore, Room schema	documents API, idempotent upload, MiniIO	Label golden set (300 docs)	Upload round trip works

Day	Android	Backend and infrastructure	AI and evaluation	Done when
4	QualityGate, OcrEngine, PdfTextExtractor	NVIDIA key, model checks, llm-smoke	Receipt JSON schema + prompt v1	OCR text visible on cards
5	DeviceTierResolver, GeminiNanoExtractor / LiteRtLmExtractor	LiteLLM gateway with NVIDIA primary + fallback models, degraded mode	eval/run_eval.py v1	First accuracy table in results/
6	Validators, TrustSignalCollector	ExtractionService (Tier C), llm_calls logging	Prompt v2 from error analysis	≥ 90% key-field accuracy
7	Chat thread UI with receipt cards and states	PolicyEngine, policy v1 JSON	Category accuracy check	Policy flags appear in app
8	Entitlement strip and line states	EntitlementService, reservations, ERP client	Planted over-limit scenarios	Exhausted broadband demo works
9	Review screen, trip grouping UI	DuplicateService, forensics service v1	Tamper recall measurement	Duplicates flagged with link
10	Question chips, answer flow	QuestionPlanner, AgentOrchestrator, tools	Questions-per-claim metric	10-receipt scenario ≤ 2 questions
11	Submit screen, status updates	ClaimStateMachine, submit to mock ERP, webhooks	Agent prompt tuning	Submit to approval end to end
12	Offline mode, SyncWorker hardening	Approver console (React) on Caddy	Latency per tier on test phone	Return-and-resubmit works
13	Polish, accessibility, empty and error states	make deploy of v0.1.0, backups, healthz	Full eval run on the NVIDIA models	Tagged release live on the domain
14	Bug bash on two devices	Load test 20-receipt batches; tune memory caps	Metrics slide from llm_calls and eval	No P0 bugs open
15	Record backup demo video	Freeze; staging snapshot	Final numbers into PRD Section 3	Demo rehearsed under 4 minutes

Definition of done for the prototype

All P0 requirements pass their acceptance criteria; the evaluation table is generated from a real run; the app works in airplane mode up to submission; the demo runs on the Oracle deployment with at least one fallback provider configured; README explains setup in under 15 minutes for a new developer.

29

AGENT-READY SPECIFICATION

Building with Claude Code

This PRD is written to be executed by an AI coding agent such as Claude Code as well as read by people. A PDF is good for review but awkward as working context, so the same content ships as a **companion kit** of plain-text files that sit in the repository root. Claude Code reads a project-level `CLAUDE.md` at the start of every session, can pull other files in with `@path imports`, and loads `CLAUDE.md` files in subdirectories when it works there. The kit uses all three.

Companion kit contents

File	Purpose for the agent
CLAUDE.md, CLAUDE.local.md	Project summary, the two workspaces, fixed stack, commands, rules (money in paise, no secrets, no direct vendor calls, no remote actions), finishing checklist. Imports the three docs below.
docs/SPEC.md	Locked decisions and every number the code needs: API request/response shapes, error codes, state transitions, thresholds, algorithms, trust weights, entitlement maths. Wins over the PDF if they differ.
docs/TASKS.md	33 task cards (T-00 to T-32) plus 9 human tasks. Each card has dependencies, PRD/SPEC references, deliverables, acceptance criteria and exact verify commands.
docs/schema.sql	The database schema from Section 23, tested on PostgreSQL 16. Alembic migration 0001 must match it.
contracts/	The shared surface between workspaces: openapi.yaml, receipt.schema.json, test vectors for both validators, the shared extraction prompt, VERSION and CHANGELOG.
docs/workspaces/	APP and BACKEND guides, setup steps for each machine, CLAUDE.local templates naming the workspace.
scripts/	Laptop helpers: Prism mock, SSH tunnel to kh-core, adb reverse for a USB phone.
data/seed/*.json	Four synthetic demo users with per-employee entitlements, and policy v1 as JsonLogic rules.
deploy/, gateway/, .env.example, .env.dev.example, Makefile	Single-server compose file, Caddyfile, database init, bootstrap, deploy, backup and smoke-test scripts, NVIDIA-only LiteLLM config. All YAML and scripts validated.
backend/CLAUDE.md, android/CLAUDE.md	Area-specific conventions loaded only when the agent works in that folder.
docs/PRD.pdf	This document, for references such as "PRD §9".

Order of authority

When sources disagree the agent follows, in order: the user's instruction in the session, **CLAUDE.local.md** (workspace), **docs/SPEC.md**, **contracts/** and **docs/schema.sql**, **docs/TASKS.md**, then this PDF. Anything still ambiguous is a question for the user, not a guess; CLAUDE.md tells the agent to stop and ask.

Working loop

1. Open a fresh session in the repository root for each task so context stays small.
2. Ask for one task: *"Read CLAUDE.md. Implement T-13 from docs/TASKS.md only. Plan first, then build, then run its Verify commands and report."*
3. Review the plan before letting it write code; check the files it intends to touch against the card's Deliver list.
4. The task is done only when every Verify command passes. The agent ticks the box in TASKS.md and summarises files changed and how they were checked.
5. Commit per task with a conventional message (for example `feat(entitlements): T-13 reservations`), push, and let CI run.
6. For HUMAN-VERIFY tasks, the agent ships code with fakes and tests; you confirm on a real phone or VM and note the result in the card.

What stays with you

Area	Why the agent does not do it	Kit support
Oracle Console, VCN, the VM	Needs your cloud login and choices about cost	bootstrap_server.sh; Section 24 steps
NVIDIA API key	Secrets must not pass through the agent	.env.example lists it; it lives only in /opt/kharcha/.env
DNS, GitHub repo and SSH key	Account-level access	H-03 and H-04 checklists
Physical phone checks	On-device GenAI, scanner and thermal behaviour need hardware	Fakes for unit tests; benchmark module; HUMAN-VERIFY steps
Deploying the hosted stack	It is the live demo on the same VM	make deploy REF=tag from /opt/kharcha; Claude Code never runs it

Guardrails written into CLAUDE.md

No secrets in code, logs or commits. No calls to model vendors except through the gateway. The LLM never decides money, limits or policy outcomes. No changes to /opt/kharcha and no deploys, ever, unless you ask. Changes stay inside the current task. Tests are written with the code, and LLM tests always run in stub mode with recorded fixtures.

Requirement traceability

Requirements (Section 6 and 9)	Tasks
ING-1 capture and multi-file	T-21
ING-2, ING-3 quality gate, PDF text layer	T-22
EXT-1, EXT-2 classify and extract	T-08, T-10, T-23
EXT-3 split multi-receipt images (PI)	T-23
EXT-4 Hindi text (PI)	T-22
EXT-5 normalisation	T-08, T-24
VER-1 trust score	T-15, T-24
VER-2 GSTIN and tax maths	T-08, T-24
VER-3 duplicates	T-14
POL-1, POL-2 policy rules and remedies	T-12, T-27
POL-3 reimbursable preview (PI)	T-23, T-27
ENT-1 to ENT-9 entitlements	T-13, T-25, T-27
AGT-1, AGT-2 grouping and completeness	T-16, T-26
AGT-3 question policy	T-17
AGT-4 natural-language edits (PI)	T-18
CLM-1, CLM-2 claim build and submission	T-16, T-19
CLM-3 approver console (PI)	T-28
CLM-4 PDF export (P2)	Backlog, after T-32

The day-by-day plan in Section 28 runs these tasks in the same order; Appendix A lists every card.

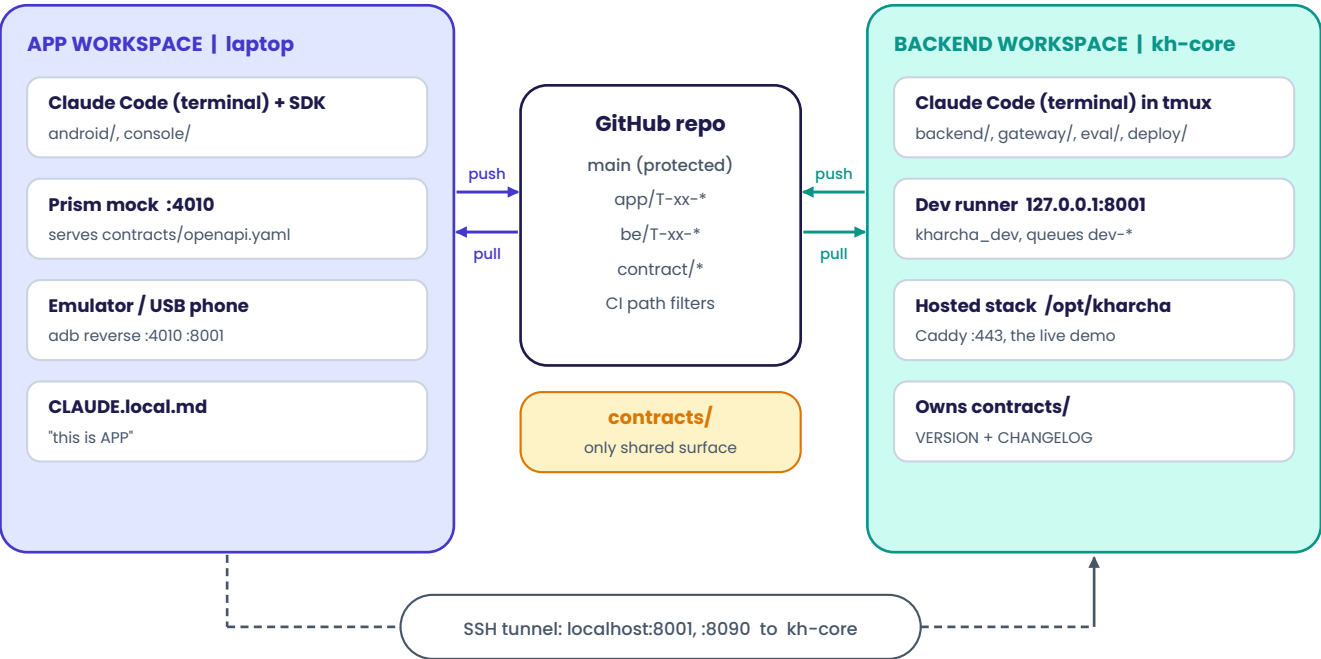
30

LAPTOP FOR THE APP, ORACLE FOR THE BACKEND

Two-workspace development

Development is split across two machines. The Android app and the approver console are built on your laptop; the backend, the AI pipeline and the LLM gateway are built on the Oracle kh-core VM, which also hosts the running backend. Claude Code is used as the terminal CLI on both machines. Both machines work on **one GitHub repository**. Each machine knows its role from an untracked `CLAUDE.local.md`, and each task card names the workspace it belongs to, so the two Claude Code sessions never edit the same folders.

TWO WORKSPACES, ONE REPOSITORY



Only Caddy (443) is public on kh-core; the dev runner binds to 127.0.0.1:8001 and is reached only through SSH.
Figure 10. The repository is the hand-off point; contracts/ is the only folder both sides depend on.

Who owns what

Workspace	Machine	Owens (may edit)	Reads only
APP	Laptop: Android SDK, JDK 17, Node.js, Claude Code CLI	android/, console/, laptop scripts	contracts/, docs/, data/demo/
BACKEND	kh-core: VM.Standard.AI.Flex, arm64, 2 OCPU, 12 GB; Claude Code in tmux; also hosts the backend	backend/, forensics/, mock-erp/, gateway/, eval/, data/, deploy/, contracts/	docs/
Shared	GitHub	main branch, CI, contracts/ releases	—

Claude Code on kh-core

Claude Code needs at least 4 GB of RAM and supports arm64, so it runs comfortably on kh-core next to the hosted stack. The bootstrap script installs it with the native installer; run it inside tmux so sessions survive SSH drops. On the laptop, Claude Code also runs in the terminal and builds the app with `./gradlew`; Android Studio is optional, for the emulator and layout previews.

The contract is the boundary

Everything the two sides must agree on lives in `contracts/`: **openapi.yaml** (every endpoint, request and response), **receipt.schema.json**, the **test vectors** both validators must pass, and the **extraction prompt** used on the phone and on the server. The backend workspace owns it. The app workspace asks for changes in `docs/CONTRACT_REQUESTS.md`. Each change goes on a `contract/` branch with a version bump, merges first, and then both sides build against it.

- **Additive changes** (new optional field) are MINOR versions; the app ignores unknown fields so it never breaks.
- **Breaking changes** are MAJOR versions and need both workspaces updated before release.
- **Backend contract test** compares FastAPI routes and models with `openapi.yaml` in CI and fails on any mismatch.
- **Prism mock** serves `openapi.yaml` on the laptop, so app work never waits for the backend.

Three environments

Environment	API runs on	App reaches it at	Use for
mock	Prism on the laptop	Emulator 10.0.2.2:4010; USB phone localhost:4010 via adb reverse	UI work from day one
dev	Dev runner on kh-core, 127.0.0.1:8001	Via SSH tunnel: 10.0.2.2:8001 or localhost:8001	Integration with the real backend
demo	Hosted stack on kh-core (Sections 24 to 25)	https://your-domain	Demo, HUMAN-VERIFY, judging

Android build flavors `mock`, `dev` and `demo` set `API_BASE_URL`. Cleartext HTTP is allowed only to localhost and 10.0.2.2 in `mock` and `dev`.

Setting up kh-core (backend workspace)

```
# after bootstrap_server.sh (Section 24) and the checkouts (Section 25 step 1)
claude # first run: complete the login link in your laptop browser
cd ~/kharcha
git sparse-checkout set --no-cone '/*' '!/android/' # optional: hide the app
make infra-up && make migrate-dev
tmux new -As kharcha # windows: claude | make dev-api | make dev-worker
curl -s 127.0.0.1:8001/v1/healthz
```

Setting up the laptop (app workspace)

```
git clone git@github.com:<you>/kharcha.git && cd kharcha
# optional: hide backend folders
git sparse-checkout set --no-cone '/*' '!/backend/' '!/forensics/' \
    '!/mock-erp/' '!/gateway/' '!/eval/'
cp docs/workspaces/CLAUDE.local.APP.md CLAUDE.local.md
./scripts/mock-api.sh # mock flavor: http://localhost:4010/v1/healthz
./scripts/tunnel.sh kh-core # dev flavor: forwards :8001 and :8090 from kh-core
./scripts/app-dev-connect.sh # USB phone: adb reverse for :4010 and :8001
claude # Claude Code in the terminal
```

Daily routine

Step	Laptop (APP)	kh-core (BACKEND)
Start	git pull --rebase; start mock or tunnel	ssh kh-core; tmux attach; git pull --rebase; make dev-api, make dev-worker
Work	One task per Claude Code session (Workspace: APP)	One task per Claude Code session (Workspace: BACKEND)
Branch	app/T-xx-topic	be/T-xx-topic or contract/topic
Finish	Verify commands, PR, CI green, merge	Verify commands, PR, CI green, merge; contract first if changed
Integrate	Switch to dev flavor after the matching backend task merges	Keep the dev runner up while the app tests against it
Release	Test the demo flavor against https://your-domain	You run make deploy REF=tag

Memory on kh-core while developing

Process	Approx. RAM	Tip
Hosted stack (Caddy, api, worker, forensics, mock ERP, gateway, Postgres, Redis, MinIO)	~5 GB caps	Always on; it is the demo
Dev runner (uvicorn --reload, rq worker)	~0.6 GB	Stop when idle
Claude Code with test runs	~1.5 to 2 GB	Close idle sessions
OS and cache	~1 to 2 GB	4 GB swap as safety net

Development never touches the demo

Dev and hosted share Postgres, Redis, MinIO and the gateway, but use separate databases, Redis DBs, buckets and queue names, and live in separate checkouts. Claude Code works only in ~/kharcha with .env.dev; the hosted stack changes only when you run make deploy. Both use the same NVIDIA key, so keep dev in stub mode unless a task needs live calls.

A

DOCS/TASKS.MD SUMMARY

Appendix: task board

Workspace: APP runs on the laptop, BACKEND on kh-core, BOTH needs both. Owner: CLAUDE means Claude Code completes and verifies the task; HUMAN-VERIFY means it builds with fakes and you confirm on hardware. Full cards with deliverables and commands are in docs/TASKS.md.

ID	Task	Workspace	Depends	Owner
T-00	Contract v1 freeze	BACKEND	—	CLAUDE
T-01	Repo scaffold and tooling	BACKEND	—	CLAUDE
T-02	Backend skeleton	BACKEND	T-01	CLAUDE
T-03	Mock ERP service	BACKEND	T-01	CLAUDE
T-04	Database models and migration	BACKEND	T-02	CLAUDE
T-05	Seed script and mock auth	BACKEND	T-03, T-04	CLAUDE
T-06	Server stack and dev runner	BACKEND	T-02, T-03	CLAUDE
T-07	Synthetic receipt generator	BACKEND	T-01	CLAUDE
T-08	Receipt contract and validators (Python)	BACKEND	T-02	CLAUDE
T-09	LLM gateway client and stub mode	BACKEND	T-04	CLAUDE
T-10	Documents API and Tier C extraction	BACKEND	T-05, T-08, T-09	CLAUDE
T-11	Evaluation harness	BACKEND	T-07, T-09	CLAUDE
T-12	Policy engine	BACKEND	T-08	CLAUDE
T-13	Entitlements and reservations	BACKEND	T-05, T-12	CLAUDE
T-14	Duplicate detection	BACKEND	T-10	CLAUDE
T-15	Forensics service and trust score	BACKEND	T-06, T-07	CLAUDE
T-16	Claim state machine and claims API	BACKEND	T-12, T-13, T-14	CLAUDE
T-17	Question planner	BACKEND	T-16	CLAUDE
T-18	Agent orchestrator and tools	BACKEND	T-09, T-17	CLAUDE
T-19	Submit, ERP sync and approvals	BACKEND	T-16, T-03	CLAUDE
T-20	Android scaffold	APP	T-01	CLAUDE
T-21	Capture, ingest, secure storage	APP	T-20	HUMAN-VERIFY
T-22	Quality gate and OCR	APP	T-21	HUMAN-VERIFY
T-23	Device tiers and extractors	APP	T-22	HUMAN-VERIFY
T-24	Validators and trust signals (Kotlin)	APP	T-20	CLAUDE
T-25	Network, sync and outbox	APP	T-20, T-00	CLAUDE
T-26	Chat thread UI	APP	T-23, T-25	HUMAN-VERIFY
T-27	Review, entitlements and submit UI	APP	T-26	HUMAN-VERIFY

ID	Task	Workspace	Depends	Owner
T-28	Approver console	APP	T-00	CLAUDE
T-29	Deployment files	BACKEND	T-06	CLAUDE
T-30	CI/CD workflows	BACKEND	T-02, T-29	CLAUDE
T-31	End-to-end demo script	BOTH	T-19, T-27, T-28	HUMAN-VERIFY
T-32	Documentation and metrics	BACKEND	T-11, T-31	CLAUDE

Human task	Before
H-01 Oracle VCN, the kh-core VM and bootstrap_server.sh	T-00
H-02 NVIDIA key into /opt/kharcha/.env	make infra-up with litellm (T-06)
H-03 DNS subdomain to kh-core	First deploy
H-04 GitHub repo, branch protection, kh-core SSH key	H-08
H-05 Confirm NVIDIA model ids, run make llm-smoke	T-09 live checks
H-06 Test phones: developer mode, GenAI availability, benchmark	T-21 to T-27 verification
H-07 First make deploy of a tag, seed, smoke test	Demo rehearsal
H-08 Claude Code login, dev and release checkouts, env files, infra-up	T-00
H-09 Laptop tools, clone, CLAUDE.local (APP), SSH config, mock and tunnel check	T-20